

American International University-Bangladesh

DEPARTMENT OF COMPUTER SCIENCE

BIG PROJECT PRACTICE GUIDE

UniPortal University Portal System

Building one ASP.NET Core solution step by step — MVC, Routing, Razor, Validation, EF Core, Session, Cookies, DTOs, AutoMapper, Authentication and Web APIs

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Coverage	Modules 0 – 12 in 14 sequential phases
Tooling	Visual Studio 2022 · SQL Server · SSMS · Postman
Instructor	Jahid Hassan

Department of Computer Science · 2026

Table of Contents

Introduction and Project Roadmap	2
Phase 0 — .NET Foundations and Environment Setup	6
Phase 1 — First MVC App: the UniPortal Skeleton	10
Phase 2 — Routing: Conventional and Attribute	16
Phase 3 — Razor Views, Layouts and Reusable UI	22
Phase 4 — Admission Form: Model Binding and Validation	28
Phase 5 — Custom Validation Attributes	35
Phase 6 — Database-First EF Core: UniversityDB	42
Phase 7 — Code-First EF Core: Courses and Enrollments	48
Phase 8 — Session and Cookies: the Registration Cart	56
Phase 9 — DTOs and AutoMapper	63
Phase 10 — Cookie Authentication and Custom Authorization	68
Phase 11 — Web API Basics: UniPortal.Api	74
Phase 12 — Full CRUD API with DTOs and AutoMapper	80
Phase 13 — Capstone: Grand Integration and Demo	87

Introduction and Project Roadmap

This practice guide turns the theory of Modules 0 to 12 of CSC4164 — Advanced Programming into one continuous, hands-on build: **UniPortal**, a University Portal System written in ASP.NET Core (.NET 8). Instead of practising each topic on throw-away examples, you will construct a single realistic product phase by phase. Every concept from your syllabus — routing, Razor views, model binding, custom validation, both EF Core approaches, session and cookie state, DTOs with AutoMapper, cookie authentication with custom authorization, and Web API development — lands in the same solution and keeps working as the portal grows around it.

The guide follows the exact structure of your CSC4164 lab manuals. Each phase contains the same eight sections you already know: Objectives, Prerequisites, Theory Recap, Tasks with Steps, Common Errors and Solutions, a Completion Checklist, Self-Assessment Questions, and Further Reading. Where a phase mirrors one of your official lab manuals (for example Phase 6 mirrors Lab 4 Database-First), the tooling is deliberately identical — Visual Studio 2022, SSMS, Package Manager Console commands, and Postman — so nothing you practise here conflicts with what your instructor will ask in class.

Warning: Important

Type all code manually. Do not copy and paste from this guide. Typing the code builds muscle memory, helps you spot errors, and improves your understanding of the syntax. This is the same rule your lab manuals enforce, and students found copy-pasting will be asked to retype.

1. How Each Phase Is Organised

Work through the phases strictly in order. Later phases extend the files you create earlier, so a skipped phase will break everything after it. Finish each phase by ticking every item in its Completion Checklist before moving on — the checklists are cumulative proof that your project is healthy at that milestone. Each task inside a phase ends with explicit test steps: run the application, visit the exact URL, and confirm the described result before continuing to the next task.

Element	Meaning in this guide
Task N	A graded unit of work inside a phase, with an estimated duration in minutes.
Step k — Title	One concrete action inside a task, usually followed by code to type.
Numbered list	Precise click-by-click instructions or test/verification steps.
Code blocks	Files to type or edits to make; the file name appears on the block header.
Warning box	A common mistake or a rule that will break the build if ignored.
Tip box	An insight your instructor may quiz you on later.
Note box	Background information that connects a task to theory.

2. What You Are Building: UniPortal

UniPortal is the online front door of a private university with two front doors of its own. The first door is an **MVC portal** used by the campus: applicants register with validated personal data, the registrar manages the department directory, students browse the course catalog and build a course registration cart, and administrators see protected dashboards. The second door is a **Web API** that exposes the same campus data as JSON for external consumers such as a future mobile app or the national university clearing house. Access is protected by cookie authentication with three roles — Admin, Faculty, and Student — enforced through a custom authorization attribute you build yourself in Phase 10.

The final solution contains two projects that share one solution file and, ultimately, one SQL Server instance. UniPortalMVC is the user-facing portal started in Phase 1 and enriched in every later phase. UniPortal.Api is the JSON service added in Phase 11 and completed in Phase 12. Data is split deliberately across two databases so that you practise both EF Core approaches exactly as the syllabus intends: UniversityDB already exists before your code does (Database-First, Phase 6), while UniPortalDb is created from your C# classes and migrations (Code-First, Phase 7).

Solution Item	Type	Purpose	Introduced In
UniPortal.sln	Solution	Container for all projects	Phase 0
UniPortalMVC	ASP.NET Core MVC (.NET 8)	Applicant registration, department directory, course catalog, registration cart, admin dashboards	Phase 1
UniPortal.Api	ASP.NET Core Web API (.NET 8)	REST JSON endpoints for departments and courses with DTOs and AutoMapper	Phase 11
UniversityDB (SQL Server)	Database — pre-existing	Legacy data: Department table; scaffolded with Database-First	Phase 6
UniPortalDb (SQL Server)	Database — generated by migrations	Code-First data: Courses, Students, Enrollments	Phase 7

3. Phase-to-Module Mapping

The table below is your contract: every syllabus module from 0 to 12 maps to exactly one phase, and the capstone phase closes the loop by integrating all of them. When you revise for Quiz or the final assessment, use this table to find the phase that practises a given topic.

Phase	Syllabus Module Covered	What You Ship at the End of the Phase
0	Module 0 — History and Evolution of .NET	Verified toolchain and the empty UniPortal solution
1	Module 1 — Introduction to ASP.NET Core MVC	UniPortalMVC project with StudentController and first views
2	Module 2 — Routing	Conventional plus attribute routes with constraints, all tested

Phase	Syllabus Module Covered	What You Ship at the End of the Phase
3	Module 3 — Views and Razor Syntax	Branded layout, partial student card, announcements view component, tag helpers
4	Module 4 — Model Binding and Validation	Applicant admission form with data annotation validation
5	Module 5 — Custom Validation	FutureDate, ConditionalRequired, UniqueEmail attributes
6	Module 6 — Database-First EF Core	UniversityDB scaffolded; Department CRUD working against SQL Server
7	Module 7 — Code-First EF Core	UniPortalDb with migrations, relationships, seed data, course CRUD
8	Module 8 — Session and Cookies	Course registration cart in session plus theme and preference cookies
9	Module 9 — DTOs and AutoMapper	Student records refactored to DTOs mapped by AutoMapper
10	Module 10 — Cookie Auth and Custom Authorization	Login, logout, roles, CustomAuthorizeAttribute protecting pages
11	Module 11 — API Basics	UniPortal.Api project with Department read endpoints in Swagger
12	Module 12 — API CRUD Development	Full Course CRUD API with DTOs, AutoMapper, status codes
13	Capstone — Grand Integration	Role-based menus, API-consuming catalog page, end-to-end registration journey

4. Databases at a Glance

Two databases, two philosophies — one campus. UniversityDB models the departments the university already had before this software project started, so you create it by hand in SSMS and let EF Core reverse-engineer the classes. UniPortalDb belongs entirely to your code: you write the entity classes first, and migrations generate the tables, foreign keys, and seed rows. Both are registered in the dependency injection container with their own connection strings, and both can live on the same local SQL Server instance.

Database	Created By	Tables	DbContext Class	Approach
UniversityDB	You, visually in SSMS	Department	UniversityDbContext	Database-First (scaffold)
UniPortalDb	Migrations (dotnet/PMC)	Course, Student, Enrollment	UniPortalDbContext	Code-First (migrations)

5. Required Software

Software	Version	Where to Get It
.NET SDK	.NET 8 or later	https://dotnet.microsoft.com/download
Visual Studio	2022 Community (free), ASP.NET and web development workload	https://visualstudio.microsoft.com
SQL Server	Express, Developer, or LocalDB	https://www.microsoft.com/sql-server
SSMS	19 or later	https://learn.microsoft.com/sql/ssms
Postman	Latest desktop version	https://www.postman.com/downloads
Browser	Chrome or Edge	Pre-installed on most systems

Tip: Before You Start

Keep all projects under one folder such as C:\AIUB\CSC4164\UniPortal. Consistent paths make the Package Manager Console commands in later phases predictable, and they mirror the folder your lab manuals use.

6. The Journey in One Picture

By Phase 13 the solution folder looks like the tree below. Folders marked with a phase number show where each piece was born; you never delete earlier work, you only extend it. Use this as a sanity check whenever your Solution Explorer feels lost — recreate the missing folder by returning to the phase that owns it.

End-state solution structure

```
C:\AIUB\CSC4164\UniPortal\ (Phase 0)
|
+-- UniPortal.sln
+-- UniPortalMVC\ (Phase 1)
| +-- Controllers\ StudentController, DepartmentController, ...
| +-- Models\ Student, AdmissionApplication, DTOs
| +-- Validations\ FutureDate, ConditionalRequired, UniqueEmail (Phase 5)
| +-- Profiles\ MappingProfile (Phase 9)
| +-- EF\Tables\ scaffolded UniversityDbContext (Phase 6)
| +-- EF\UniPortalDbContext.cs (Phase 7)
| +-- Views\ Razor views per controller
| +-- wwwroot\ static files
| +-- Program.cs services and middleware pipeline
+-- UniPortal.Api\ (Phase 11)
+-- Controllers\ DepartmentsController, CoursesController
+-- DTOs\ CourseApiDTO, CourseSavedDTO (Phase 12)
+-- Profiles\ MappingProfile (API side) (Phase 12)
+-- EF\ scaffolded contexts for both databases
+-- Program.cs
```

Phase 0 — .NET Foundations and Environment Setup

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 0 — .NET Foundations and Environment Setup
Syllabus Module	Module 0 — History and Evolution of .NET
Instructor	Jahid Hassan
Duration	90 minutes (self-paced, before Phase 1)
Work Mode	Individual — project-based practice
Project	MediCare — Hospital Management System
Prerequisite	None — this is the starting point of the UniPortal project

1. Phase Objectives

By the end of this phase, you will be able to:

- Trace the evolution of .NET from .NET Framework (2002) to .NET 8 and explain why ASP.NET Core was rebuilt from scratch
- Install and verify the .NET 8 SDK, Visual Studio 2022, SQL Server, SSMS, and Postman
- Create a blank solution named UniPortal that will host every later phase
- Use the Developer PowerShell to confirm the installed SDK version

2. Prerequisites

- A Windows PC with at least 8 GB RAM and 20 GB of free disk space
- Administrator access to install software
- You have watched or read Module 0 (History of .NET) from the theory class

3. Theory Recap

3.1 From .NET Framework to .NET 8

Microsoft shipped the first .NET Framework in 2002 together with ASP.NET Web Forms. It ran only on Windows, was installed machine-wide, and version upgrades were risky because the runtime was shared by every application on the computer. ASP.NET MVC arrived in 2009 and brought the Model-View-Controller pattern to the Microsoft web stack, but it still sat on the same Windows-only runtime. In 2016 Microsoft

responded with .NET Core: a cross-platform, open-source, side-by-side installable runtime with a brand-new, fast web framework called ASP.NET Core. Since .NET 5 (2020) the separate lines merged into a single .NET, and .NET 8 (2023) is the current Long Term Support release this guide targets.

Year	Release	Why It Matters for UniPortal
2002	.NET Framework 1.0 and ASP.NET Web Forms	Server controls hid HTML; the pattern this course replaces
2009	ASP.NET MVC	Introduced Model-View-Controller separation to .NET web apps
2016	.NET Core 1.0 and ASP.NET Core	Cross-platform, open source, modular, high performance
2019	.NET Core 3.x	Last Core-branded release; EF Core matured
2020	.NET 5	Framework and Core unified under one name
2021	.NET 6 (LTS)	Minimal hosting model — the Program.cs you will edit
2022	.NET 7	Performance and native AOT improvements
2023	.NET 8 (LTS)	Target runtime for UniPortal — supported until 2026

3.2 .NET Framework vs .NET Core/.NET 8

Aspect	.NET Framework	.NET 8 (Core)
Platforms	Windows only	Windows, Linux, macOS, containers
Delivery	Machine-wide installer	Per-app, side-by-side runtimes
Open source	Partial	Fully open source on GitHub
Web stack	ASP.NET (System.Web)	ASP.NET Core — Kestrel, middleware pipeline
Future	Maintenance mode only	All new development happens here

Note: Key point

The MVC pattern you practise in Phase 1 is 15 years old, but the runtime under it was rebuilt in 2016. When your instructor asks "why ASP.NET Core matters", the short answer is: cross-platform, faster, modular, open source, and the only road forward.

4. Phase Tasks

Warning: Important

Type all commands manually. Do not copy and paste from this guide.

Task 1 Install the Development Toolchain (40 min)

Step 1 — Install the required software

1. Download and install the **.NET SDK 8** (not just the runtime) from <https://dotnet.microsoft.com/download>
2. Install **Visual Studio 2022 Community** and make sure the workload **ASP.NET and web development** is ticked during installation
3. Install **SQL Server Express or Developer Edition**, then install **SQL Server Management Studio (SSMS)**
4. Install the **Postman** desktop application for the API phases

Warning: Workload check

If the ASP.NET and web development workload is missing, open the Visual Studio Installer, click Modify, tick the workload, and install. Without it the MVC and Web API project templates will not exist in Phase 1 and Phase 11.

Task 2 Verify the SDK from the Command Line (15 min)

Step 1 — Open the developer shell and check versions

1. Open **Developer PowerShell** (search "Developer PowerShell for VS 2022")
2. Run the commands below and confirm the version reported starts with 8

Developer PowerShell

```
dotnet --info  
dotnet --version
```

Tip: Reading the output

`dotnet --info` lists every installed runtime and SDK. If you see several SDKs, that is fine — .NET installs side by side. Visual Studio simply uses the newest one when the project file targets `net8.0`.

Task 3 Create the UniPortal Blank Solution (20 min)

Step 1 — Create the solution in Visual Studio

1. Open Visual Studio and click **Create a new project**
2. Search for **Blank Solution**, select it, and click Next
3. Set the Solution name to: UniPortal
4. Set the location to: C:\AIUB\CSC4164\
5. Click Create — you now have an empty solution that Phases 1 and 11 will fill

Step 2 — Confirm the folder layout

1. Open File Explorer and verify the folder C:\AIUB\CSC4164\UniPortal exists
2. Inside it you should see exactly one file: UniPortal.sln

5. Common Errors and Solutions

Error	Likely Cause	Solution
dotnet is not recognised	SDK not installed or PATH not refreshed	Install the SDK and reopen the developer shell
MVC template missing in Visual Studio	ASP.NET workload not installed	Visual Studio Installer, Modify, tick ASP.NET and web development
SSMS cannot connect to localhost	SQL Server service stopped	Start the SQL Server (SQLEXPRESS) service in services.msc
Solution name typo	Created project instead of blank solution	Delete and recreate, choosing the Blank Solution template

6. Completion Checklist

#	Checklist Item	Done
1	.NET SDK 8 installed and dotnet --version reports 8.x	☐
2	Visual Studio 2022 with the ASP.NET and web development workload	☐
3	SQL Server and SSMS installed and connecting	☐
4	Postman installed for the API phases	☐
5	Blank solution UniPortal.sln created under C:\AIUB\CSC4164\	☐
6	You can explain Framework vs Core in one sentence each	☐

7. Self-Assessment Questions

- What were the three main limitations of .NET Framework that led to .NET Core?
- What does CLR stand for, and which of its responsibilities survive in .NET 8?
- Why can multiple .NET runtimes coexist on one machine without conflicts?
- In which year did the Framework and Core lines merge, and under what name?
- Give two reasons a university would prefer .NET 8 over .NET Framework 4.8 for a new student portal.

8. Further Reading

- .NET history and versions: <https://learn.microsoft.com/dotnet/core/introduction>
- Choose between .NET Framework and .NET:
<https://learn.microsoft.com/dotnet/standard/choosing-core-framework-server>

End of Phase — next: Phase 1 creates the UniPortalMVC project — your first MVC application.

Phase 1 — First MVC App: the UniPortal Skeleton

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 1 — First MVC App: the UniPortal Skeleton
Syllabus Module	Module 1 — Introduction to ASP.NET Core MVC
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual — project-based practice
Project	MediCare — Hospital Management System
Prerequisite	Phase 0 completed — toolchain installed and UniPortal solution created

1. Phase Objectives

By the end of this phase, you will be able to:

- Describe the Model-View-Controller responsibilities and the request-response lifecycle of ASP.NET Core MVC
- Identify every folder of an MVC project and the role of Program.cs
- Create an MVC project inside an existing solution
- Write a controller with multiple actions and strongly-typed views
- Run the app and trace one request through the Network tab

2. Prerequisites

- Phase 0 completed — UniPortal.sln exists under C:\AIUB\CSC4164\
- Theory class Module 1 (Introduction to ASP.NET Core MVC) attended

3. Theory Recap

3.1 MVC Architecture in UniPortal

Layer	Folder	UniPortal Example
Model	Models/	Student class carrying Id, name, program, CGPA
View	Views/Student/	Index.cshtml listing students as an HTML table
Controller	Controllers/	StudentController deciding which view to render

When a student opens /Student/Details/2, the middleware pipeline passes the request to routing, routing selects StudentController.Details, the action fetches the model and returns View(model), Razor converts the view into HTML, and the response travels back to the browser. Every phase of this guide plays somewhere inside that pipeline.

3.2 Program.cs — the Composition Root

Program.cs (template, annotated)

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews(); // MVC services

var app = builder.Build();
app.UseStaticFiles(); // wwwroot css/js/images
app.UseRouting(); // decide the endpoint from the URL
app.MapControllerRoute( // convention: {controller}/{action}/{id?}
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

Note: Key point

The URL pattern /Controller/Action/Id maps directly to the controller class name, the action method name, and the route parameter — the exact behaviour you will reconfigure in Phase 2.

4. Phase Tasks

Warning: Important

Type all code manually. Every file below is part of the final portal.

Task 1 Create UniPortalMVC Inside the Solution (15 min)

Step 1 — Add the project

1. Right-click the UniPortal solution, Add, then **New Project**
2. Choose **ASP.NET Core Web App (Model-View-Controller)**, click Next
3. Project name: UniPortalMVC — location stays inside the solution folder
4. Framework: **.NET 8.0 (Long Term Support)**, Authentication type: **None**, Configure for HTTPS: ticked, then Create

Task 2 Model and Controller (30 min)

Step 1 — Create the Student model

Models/Student.cs

```
namespace UniPortalMVC.Models
{
```

```
public class Student
{
    public int StudentId { get; set; }
    public string FullName { get; set; }
    public string Program { get; set; }
    public double Cgpa { get; set; }
}
}
```

Step 2 — Create StudentController

1. Right-click Controllers, Add, Controller, then **MVC Controller — Empty**
2. Name it StudentController and type the code below

Controllers/StudentController.cs

```
using Microsoft.AspNetCore.Mvc;
using UniPortalMVC.Models;

namespace UniPortalMVC.Controllers
{
    public class StudentController : Controller
    {
        private static readonly List<Student> _students = new()
        {
            new Student { StudentId = 1, FullName = "Tasin Rahman",
                Program = "CSE", Cgpa = 3.75 },
            new Student { StudentId = 2, FullName = "Nusrat Jahan",
                Program = "EEE", Cgpa = 3.90 },
            new Student { StudentId = 3, FullName = "Rafid Hasan",
                Program = "BBA", Cgpa = 3.40 }
        };

        public IActionResult Index()
        {
            return View(_students);
        }

        public IActionResult Details(int? id)
        {
            if (id == null) return NotFound();
            var student = _students.FirstOrDefault(s => s.StudentId == id);
            if (student == null) return NotFound();
            return View(student);
        }

        public IActionResult About()
        {
            return Content("UniPortal student services portal");
        }
    }
}
```

Tip: Why static?

The list is static so it survives between requests. A new list would be created for every request and Details would work while Index seemed to forget students — a classic runtime surprise.

Task 3 Views for Index and Details (35 min)

Step 1 — Create the folder and Index view

1. Right-click Views, Add, New Folder, name it Student
2. Right-click the Student folder, Add, View, Razor View — Empty, name Index

Views/Student/Index.cshtml

```
@model List<UniPortalMVC.Models.Student>
@{ ViewData["Title"] = "Student Directory"; }

<h2 class="mb-3">Student Directory</h2>

@if (Model.Count == 0)
{
    <p>No students registered yet.</p>
}
else
{
    <table class="table table-striped">
    <thead>
    <tr>
    <th>ID</th>
    <th>Full Name</th>
    <th>Program</th>
    <th>CGPA</th>
    </tr>
    </thead>
    <tbody>
    @foreach (var s in Model)
    {
    <tr>
    <td>@s.StudentId</td>
    <td>@s.FullName</td>
    <td>@s.Program</td>
    <td>@s.Cgpa.ToString("0.00")</td>
    </tr>
    }
    </tbody>
    </table>
}
```

Step 2 — Create the Details view

Views/Student/Details.cshtml

```
@model UniPortalMVC.Models.Student
@{ ViewData["Title"] = "Student Profile"; }

<h2>@Model.FullName</h2>
```

```
<dl class="row">
  <dt class="col-sm-3">Student ID</dt>
  <dd class="col-sm-9">@Model.StudentId</dd>
  <dt class="col-sm-3">Program</dt>
  <dd class="col-sm-9">@Model.Program</dd>
  <dt class="col-sm-3">CGPA</dt>
  <dd class="col-sm-9">@Model.Cgpa.ToString("0.00")</dd>
</dl>
<a asp-controller="Student" asp-action="Index" class="btn btn-secondary">
  Back to Directory
</a>
```

Task 4 Brand the Shared Layout (25 min)

Open Views/Shared/_Layout.cshtml and personalise the template: replace the site name with **UniPortal**, keep the navbar, and add a Students menu item.

Views/Shared/_Layout.cshtml (navbar)

```
<nav class="navbar navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" asp-controller="Home"
      asp-action="Index">UniPortal</a>
    <ul class="navbar-nav flex-row gap-3">
      <li class="nav-item">
        <a class="nav-link" asp-controller="Student"
          asp-action="Index">Students</a>
      </li>
    </ul>
  </div>
</nav>
```

Step 2 — Update the footer

Views/Shared/_Layout.cshtml (footer)

```
<footer class="border-top footer text-muted">
  <div class="container">
    &copy; @DateTime.Now.Year - UniPortal, Registrar Office
  </div>
</footer>
```

Task 5 Observe the Request-Response Cycle (15 min)

1. Run the app with F5 and open Developer Tools with F12
2. Open the **Network** tab, then navigate to /Student
3. Confirm the request is GET with status 200 and content type text/html
4. Navigate to /Student/Details/2 and confirm the profile of Nusrat Jahan
5. Navigate to /Student/Details/99 and confirm the 404 response
6. Navigate to /Student/About and confirm plain text without any layout

5. Common Errors and Solutions

Error	Likely Cause	Solution
View not found	View saved in the wrong folder or wrong name	Views must be in Views/Student/ named Index.cshtml and Details.cshtml
Build error: List not found	Missing using directive	The C# 12 implicit usings cover System.Collections.Generic; check ImplicitUsings is enabled in the csproj
Details returns empty page	Id not matched	Confirm the URL is /Student/Details/2 and the route value name is id
Layout still shows template menu	Edited the wrong layout file	Edit Views/Shared/_Layout.cshtml used by _ViewStart.cshtml

6. Completion Checklist

#	Checklist Item	Done
1	UniPortalMVC project created with .NET 8 and Authentication None	☐
2	Student model created with four properties	☐
3	StudentController serves Index, Details, and About actions	☐
4	Static in-memory student list returns three students	☐
5	Index view renders a Bootstrap table of students	☐
6	Details view shows one student and 404s on unknown ids	☐
7	Layout carries the UniPortal brand and the Students menu item	☐
8	Request-response cycle observed in the Network tab	☐

7. Self-Assessment Questions

- What is the role of the Controller in the MVC pattern?
- What does `return View(_students)` pass to the view, and how is it received?
- What is the difference between `ViewBag` and `ViewData`?
- Why must the student list be static for this demo to work?
- Where must a Razor view be placed so the framework finds it automatically?

8. Further Reading

- Controllers in ASP.NET Core MVC: <https://learn.microsoft.com/aspnet/core/mvc/controllers/actions>
- Views in ASP.NET Core MVC: <https://learn.microsoft.com/aspnet/core/mvc/views/overview>

- Add a controller to an ASP.NET Core MVC app:
<https://learn.microsoft.com/aspnet/core/tutorials/first-mvc-app/adding-controller>

End of Phase — next: Phase 2 rewires how UniPortal URLs are matched — Routing.

Phase 2 — Routing: Conventional and Attribute

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 2 — Routing: Conventional and Attribute
Syllabus Module	Module 2 — Routing in ASP.NET Core MVC
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual — project-based practice
Project	MediCare — Hospital Management System
Prerequisite	Phase 1 completed — UniPortalMVC runs with the student directory

1. Phase Objectives

By the end of this phase, you will be able to:

- Explain the default conventional route pattern token by token
- Add custom conventional routes with defaults and inline constraints
- Apply attribute routing with [Route] at controller and action level
- Use the [controller] and [action] tokens in route templates
- Predict which route wins when several patterns match the same URL

2. Prerequisites

- Phase 1 completed — StudentController and views are working
- Theory class Module 2 (Routing) attended

3. Theory Recap

3.1 Two Routing Styles

Style	Where Defined	Example
Conventional	Program.cs — applies globally	/Student/Details/5 maps to StudentController.Details(5)
Attribute	Directly on controllers and actions with [Route]	[Route("students/{id:int}")] above the action method

3.2 Anatomy of the Default Pattern

Program.cs (default route)

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

// {controller=Home} -> if URL segment missing, use HomeController
// {action=Index} -> if URL segment missing, use Index()
// {id?} -> optional parameter, binds to int id when present
```

3.3 Common Route Constraints

Constraint	Meaning	Example Match
:int	Integer only	/Student/Details/5
:alpha	Letters only	/campus/library
:min(1)	Minimum value	{id:min(1)} rejects 0 and negatives
:maxlength(10)	Maximum length	Segment of at most 10 characters
:datetime	Valid date	/report/2026-01-31

4. Phase Tasks

Warning: Important

Type all code manually. Every routing change requires a rebuild before the browser test reflects it.

Task 1 Verify the Default Route Behaviour (10 min)

1. Run the app and confirm `http://localhost:port/` opens Home/Index
2. Navigate to `/Student` and confirm it reaches StudentController.Index
3. Navigate to `/Student/Details/1` and confirm the profile view
4. Write down which segments were explicit and which came from defaults

Task 2 Add a Custom Conventional Route with Constraints (25 min)

Step 1 — Add the campus route above the default route

Open Program.cs and add this route before MapControllerRoute named "default":

```
Program.cs

app.MapControllerRoute(
    name: "campus",
    pattern: "campus/{building:alpha}/{room:int?}",
    defaults: new { controller = "Campus", action = "Info" });

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Step 2 — Create CampusController

1. Add an empty MVC controller named CampusController and type:

```
Controllers/CampusController.cs

using Microsoft.AspNetCore.Mvc;

namespace UniPortalMVC.Controllers
{
    public class CampusController : Controller
    {
        // GET /campus/library or /campus/library/3
        public IActionResult Info(string building, int? room)
        {
            ViewBag.Building = building ?? "main";
            ViewBag.Room = room ?? 1;
            return View();
        }
    }
}
```

Step 3 — Create Views/Campus/Info.cshtml

```
Views/Campus/Info.cshtml

@{ ViewData["Title"] = "Campus Guide"; }

<h2>Building: @ViewBag.Building</h2>
<p>The service desk sits in room <strong>@ViewBag.Room</strong>.</p>
```

Step 4 — Test the route and its constraints

1. Navigate to /campus/library — building library, room 1 (default)
2. Navigate to /campus/library/3 — room now 3
3. Navigate to /campus/library/abc — 404, because :int rejects abc
4. Navigate to /campus/123/2 — 404, because :alpha rejects digits

Tip: Route order matters

More specific routes must be registered above less specific ones. If you place the campus route after the default route, the default pattern would never let "campus" reach CampusController.Info.

Task 3 Attribute Routing on the Student Directory (30 min)

Step 1 — Decorate StudentController with routes

Modify StudentController so the class and each action carry explicit route templates:

```
Controllers/StudentController.cs (attribute routing)

using Microsoft.AspNetCore.Mvc;
using UniPortalMVC.Models;

namespace UniPortalMVC.Controllers
{
    [Route("students")]
    public class StudentController : Controller
    {
        private static readonly List<Student> _students = new()
        {
            new Student { StudentId = 1, FullName = "Tasin Rahman",
                Program = "CSE", Cgpa = 3.75 },
            new Student { StudentId = 2, FullName = "Nusrat Jahan",
                Program = "EEE", Cgpa = 3.90 },
            new Student { StudentId = 3, FullName = "Rafid Hasan",
                Program = "BBA", Cgpa = 3.40 }
        };

        // GET /students
        [Route("")]
        [Route("all")] // second route to the same action
        public IActionResult Index()
        {
            ViewBag.Title = "Find a Student";
            return View(_students);
        }

        // GET /students/2
        [Route("{id:int:min(1)}")]
        public IActionResult Details(int id)
        {
            var student = _students.FirstOrDefault(s => s.StudentId == id);
            if (student == null) return NotFound();
            return View(student);
        }

        // GET /students/search?keyword=cse
        [Route("search")]
        public IActionResult Search(string keyword)
        {
            ViewBag.Keyword = keyword;
            return View();
        }
    }
}
```

```
}
}
}
```

Step 2 — Create the Search view

Views/Student/Search.cshtml

```
@{ ViewData["Title"] = "Search Students"; }

<h2>Student Search</h2>
<p>Keyword received: <strong>@ViewBag.Keyword</strong></p>
```

Step 3 — Test attribute URLs

1. /students and /students/all both render the directory
2. /students/2 renders Nusrat Jahan; /students/99 returns 404
3. /students/search?keyword=cse shows the keyword from the query string
4. The old /Student/Details/2 URL now returns 404 — attribute routes replaced it

Note: Notice the difference

With attribute routing the URL is /students/2, not /Student/Details/2. The [Route] attribute fully controls the URL shape, while the conventional route keeps working for controllers without attributes.

Task 4 Use the [controller] and [action] Tokens (15 min)

Step 1 — Add a token-based route to CampusController

Controllers/CampusController.cs (token route)

```
[Route("services/[controller]/[action]")]
public class CampusController : Controller
{
    // GET /services/Campus/Info
    public IActionResult Info(string building, int? room)
    {
        ViewBag.Building = building ?? "main";
        ViewBag.Room = room ?? 1;
        return View();
    }
}
```

1. Navigate to /services/Campus/Info — the tokens resolved to the class and action names
2. Rename awareness: if the class were ever renamed, the URL follows automatically — one benefit of token templates

Task 5 Route Testing Matrix (20 min)

Test every URL below and fill in your observation column in your notebook — reading route patterns and predicting behaviour is a common quiz topic.

URL to Test	Expected Result	Your Observation
/	Home/Index via default route	
/Student	404 — attribute route replaced conventional	
/students	Student directory	
/students/all	Same directory via second attribute route	
/students/2	Nusrat Jahan profile	
/students/0	404 — :min(1) rejects zero	
/campus/library/3	Building library, room 3	
/campus/123/3	404 — :alpha rejects digits	

5. Common Errors and Solutions

Error	Likely Cause	Solution
404 on a working action	URL does not match any route template	Check the [Route] values and remember the Controller suffix is dropped
AmbiguousMatchException	Two routes match the same URL	Remove one route or make templates distinguishable
Constraint never rejects values	Constraint typo such as :INT	Constraints are lowercase: :int, :alpha, :min(1)
Token route keeps literal name	Used [Controller] capital C	Tokens are case-sensitive: [controller] and [action]
Old links break after attribute routing	Expected both styles to work	A controller with [Route] ignores the conventional route by design

6. Completion Checklist

#	Checklist Item	Done
1	Campus conventional route with :alpha and optional :int works	☐
2	Non-integer room and digit-only building both confirmed to return 404	☐
3	StudentController fully moved to attribute routing	☐
4	/students, /students/all, /students/2 and /students/search all respond	☐
5	Token route /services/Campus/Info resolves correctly	☐
6	All 8 URLs of the testing matrix tested and observations recorded	☐

7. Self-Assessment Questions

- What is the difference between conventional routing and attribute routing? When would you use each?
- What does the {id?} token mean in a route pattern?
- If a route has the constraint {id:int}, what happens when a non-integer value is passed?
- Why must the campus route be registered before the default route?
- What do the [controller] and [action] tokens resolve to, and why is that useful during refactoring?

8. Further Reading

- Routing in ASP.NET Core: <https://learn.microsoft.com/aspnet/core/fundamentals/routing>
- Route constraints reference:
<https://learn.microsoft.com/aspnet/core/fundamentals/routing#route-constraints>

End of Phase — next: Phase 3 teaches Razor views, layouts and reusable UI.

Phase 3 — Razor Views, Layouts and Reusable UI

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 3 — Razor Views, Layouts and Reusable UI
Syllabus Module	Module 3 — Views and Razor Syntax
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual — project-based practice
Project	MediCare — Hospital Management System
Prerequisite	Phase 2 completed — attribute routing active on /students

1. Phase Objectives

By the end of this phase, you will be able to:

- Write Razor expressions, code blocks, conditionals and loops
- Pass data with ViewBag, ViewData and strongly-typed @model
- Brand the shared layout and define optional named sections
- Extract a repeating card into a partial view
- Build a view component and invoke it from any view

- Use anchor, image, form, input and validation tag helpers

2. Prerequisites

- Phase 2 completed — /students directory running
- Theory class Module 3 (Views and Razor Syntax) attended

3. Theory Recap

3.1 Razor Syntax Quick Reference

Syntax	Purpose	Example
@variable	Output a C# variable inline in HTML	@ViewBag.Title
@{ }	C# code block with no output	@{ var count = 5; }
@if () { }	Conditional rendering	@if (Model != null) { Found }
@foreach () { }	Loop over a collection	@foreach (var s in Model) { @s }

3.2 Reusability Building Blocks

Tool	Lives In	Use It For
Layout	Views/Shared/_Layout.cshtml	Page chrome shared by every view
Section	@RenderSection in layout	Optional per-page scripts or styles
Partial view	Views/Shared/_StudentCard.cshtml	Repeating markup fragment with a model
View component	C# class + Default.cshtml	Self-contained widget with logic, e.g. announcements

Note: Tag helpers vs HTML helpers

Tag helpers look like normal HTML attributes (asp-for, asp-action) and are processed on the server. They power every form you will build in Phases 4 and 5. Enable them once in Views/_ViewImports.cshtml with @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers.

4. Phase Tasks

Warning: Important

Type all code manually. Razor is case-sensitive: @model is not @Model.

Task 1 Sharpen Razor in the Directory View (20 min)

Step 1 — Extend Views/Students/Index.cshtml

Views/Students/Index.cshtml (excerpt)

```
@model List<UniPortalMVC.Models.Student>
@{ ViewData["Title"] = "Student Directory"; }

<h2>Student Directory <small class="text-muted">@Model.Count records</small></h2>

@foreach (var s in Model)
{
    <p>
        @s.FullName —
        @if (s.Cgpa >= 3.75)
        {
            <span class="badge bg-success">High Distinction</span>
        }
        else
        {
            <span class="badge bg-secondary">Good Standing</span>
        }
    </p>
}
```

1. Run the app and confirm the badges render per student
2. View page source and confirm the C# never leaks into the HTML

Task 2 Layout, Sections and Navigation (30 min)

Step 1 — Finalise the UniPortal navbar

Views/Shared/_Layout.cshtml (navbar)

```
<nav class="navbar navbar-expand-sm navbar-dark bg-dark">
    <div class="container">
        <a class="navbar-brand" asp-controller="Home" asp-action="Index">UniPortal</a>
        <ul class="navbar-nav flex-row gap-3">
            <li class="nav-item">
                <a class="nav-link" asp-controller="students" asp-action="">Directory</a>
            </li>
            <li class="nav-item">
                <a class="nav-link" asp-controller="Campus" asp-action="Info">Campus</a>
            </li>
        </ul>
    </div>
</nav>
```

Step 2 — Define a scripts section in the layout

Views/Shared/_Layout.cshtml (body end)

```
@RenderBody()
<footer class="border-top footer text-muted">
    <div class="container">&copy; @DateTime.Now.Year - UniPortal</div>
</footer>
<script src="~/lib/jquery/dist/jquery.min.js"></script>
```

```
@await RenderSectionAsync("Scripts", required: false)
```

Any view can now inject page-specific JavaScript into the Scripts section without touching the layout — the announcement component in Task 4 uses exactly this hook.

Tip: required: false

With required: false, views without a Scripts section simply skip it. With required: true, every view that omits the section throws at runtime.

Task 3 Extract the _StudentCard Partial (30 min)

Step 1 — Create the partial view

Views/Shared/_StudentCard.cshtml

```
@model UniPortalMVC.Models.Student
<div class="card mb-2" style="max-width: 26rem;">
<div class="card-body">
<h5 class="card-title">@Model.FullName</h5>
<h6 class="card-subtitle text-muted">@Model.Program</h6>
<p class="card-text">CGPA @Model.Cgpa.ToString("0.00")</p>
<a asp-controller="students" asp-action="Details"
  asp-route-id="@Model.StudentId" class="card-link">Profile</a>
</div>
</div>
```

Step 2 — Render it from the directory

Views/Students/Index.cshtml (cards)

```
@model List<UniPortalMVC.Models.Student>

<div class="row">
@foreach (var s in Model)
{
<div class="col-md-4">
<partial name="_StudentCard" model="s" />
</div>
}
</div>
```

1. Run and confirm three bootstrap cards, each linking to /students/{id}

Task 4 Build the AnnouncementBoard View Component (35 min)

Step 1 — Create the component class

1. Add a folder ViewComponents at the project root
2. The class must end with ViewComponent and derive from ViewComponent

ViewComponents/AnnouncementBoard.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
namespace UniPortalMVC.ViewComponents
{
    public class AnnouncementBoard : ViewComponent
    {
        private static readonly List<(string Text, string Level)> _items = new()
        {
            ("Midterm routine published for Spring 2026", "info"),
            ("Tuition waiver applications close on 20 March", "warning"),
            ("Library extends hours during exam week", "info")
        };

        public IViewComponentResult Invoke(int max = 3)
        {
            var model = _items.Take(max).ToList();
            return View(model);
        }
    }
}
```

Step 2 — Create its view

1. Create the exact path Views/Shared/Components/AnnouncementBoard/Default.cshtml

Views/Shared/Components/AnnouncementBoard/Default.cshtml

```
@model List<(string Text, string Level)>
<div class="card border-primary mb-3">
<div class="card-header bg-primary text-white">Notice Board</div>
<ul class="list-group list-group-flush">
@foreach (var item in Model)
{
<li class="list-group-item">
@if (item.Level == "warning")
{
<span class="badge bg-warning text-dark">Deadline</span>
}
else
{
<span class="badge bg-info text-dark">Info</span>
}
@item.Text
</li>
}
</ul>
</div>
```

Step 3 — Invoke it on the Home view

Views/Home/Index.cshtml (invoke)

```
<vc:announcement-board max="2" />
```

1. Run the app — the Home page shows two announcements
2. Change max to 3, rebuild, confirm three rows — the Invoke parameter flows from the markup to C#

Note: Why a component and not a partial?

A partial only displays what you pass to it. A view component fetches or computes its own data (here the notice list) — the right tool when the widget owns its logic.

Task 5 Tag Helper Sweep on the Directory (25 min)

Step 1 — Confirm tag helpers are imported

```
Views/_ViewImports.cshtml (already in template)
@using UniPortalMVC
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Step 2 — Convert one raw link and one raw image

```
Tag helper conversion example
<!-- plain HTML -->
<a href="/Students/Details/2">Nusrat</a>

<!-- same link with tag helpers: URL generated by the framework -->
<a asp-controller="students" asp-action="Details" asp-route-id="2">Nusrat</a>
```

1. Run, right-click the generated link, Inspect — confirm the href points to /Students/Details/2
2. Rename the Details action to Profile, rebuild, and confirm the tag helper version updates while the hardcoded href 404s — that is the maintenance win

5. Common Errors and Solutions

Error	Likely Cause	Solution
asp-* attributes ignored	Tag helpers not added in _ViewImports	Add @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
Partial renders empty	model kwarg missing on partial tag	Use model="s" — name="..." alone passes no data
View component not found	Wrong folder or class name	Class must end in ViewComponent; view lives in Views/Shared/Components/{Name}/Default.cshtml
Section never renders	Forgot @await RenderSectionAsync	Add the render call to _Layout.cshtml; keep required: false
@Model is null	View opened without a model parameter	Pass the model from the action: return View(student)

6. Completion Checklist

#	Checklist Item	Done
1	Directory view shows per-student badges via @if	☐

#	Checklist Item	Done
2	Layout navbar links Directory and Campus with tag helpers	☐
3	Scripts section defined with required: false and used by one view	☐
4	_StudentCard partial renders three linked cards	☐
5	AnnouncementBoard component renders with the max parameter honoured	☐
6	One hardcoded link converted to tag helpers and verified in the browser	☐

7. Self-Assessment Questions

- What does @RenderBody() do in a layout file?
- What does required: false mean in @RenderSection?
- When would you choose a view component over a partial view?
- How does asp-route-id change the generated URL?
- Where must a partial view used by many controllers live?

8. Further Reading

- Razor syntax reference: <https://learn.microsoft.com/aspnet/core/mvc/views/razor>
- Layout in ASP.NET Core MVC: <https://learn.microsoft.com/aspnet/core/mvc/views/layout>
- View components: <https://learn.microsoft.com/aspnet/core/mvc/views/view-components>
- Tag helpers: <https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/intro>

End of Phase — next: Phase 4 builds the validated admission form — model binding.

Phase 4 — Admission Form: Model Binding and Validation

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 4 — Admission Form: Model Binding and Validation
Syllabus Module	Module 4 — Model Binding and Validation
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual — project-based practice

Project	MediCare — Hospital Management System
Prerequisite	Phase 3 completed — layout, partials and tag helpers working

1. Phase Objectives

By the end of this phase, you will be able to:

- Explain how ASP.NET Core binds request data to action parameters
- Choose the right binding source: query, route, form, header
- Validate input with data annotations on a model class
- Render inline errors with asp-validation-for and the client scripts partial
- Guard every POST with ModelState.IsValid and understand why

2. Prerequisites

- Phase 3 completed — tag helpers enabled in `_ViewImports`
- Theory class Module 4 (Model Binding and Validation) attended

3. Theory Recap

3.1 Binding Sources

Source	Attribute	Typical Use in UniPortal
Form fields	[FromForm] (default for POST)	Admission application fields
Route values	[FromRoute]	Student id in <code>/students/5</code>
Query string	[FromQuery]	Search keyword <code>?keyword=cse</code>
Request header	[FromHeader]	Tracking id sent by an integration
Body JSON	[FromBody]	Web API endpoints (Phase 11-12)

3.2 Data Annotations You Will Type Today

Attribute	Validates	Example
[Required]	Value present	Applicant name
[StringLength(60, MinimumLength = 3)]	Length window	Name length
[EmailAddress]	Email shape	Applicant email
[RegularExpression]	Pattern match	Bangladeshi mobile number
[Range(0.0, 4.0)]	Numeric bounds	School CGPA

Attribute	Validates	Example
[Compare]	Two properties match	Password confirmation

3.3 The ModelState Workflow

POST-Redirect-GET with validation guard

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Apply(AdmissionApplication form)
{
    if (!ModelState.IsValid) // server-side re-check of every annotation
    {
        return View(form); // redisplay with inline errors
    }
    // ... accept the application ...
    return RedirectToAction("Success");
}
```

Warning: Never trust the browser

Client validation can be switched off by any user. `ModelState.IsValid` is the real gate; the jQuery unobtrusive scripts are only a convenience layer on top of the same annotations.

4. Phase Tasks

Warning: Important

Type all code manually. Attribute typos fail silently — a misspelled `[Required]` is just a string to the compiler.

Task 1 Create the AdmissionApplication Model (30 min)

Step 1 — Type the model with annotations

Models/AdmissionApplication.cs

```
using System.ComponentModel.DataAnnotations;

namespace UniPortalMVC.Models
{
    public class AdmissionApplication
    {
        [Required(ErrorMessage = "Applicant name is required")]
        [StringLength(60, MinimumLength = 3)]
        [Display(Name = "Full name")]
        public string ApplicantName { get; set; }

        [Required(ErrorMessage = "Email is required")]
        [EmailAddress(ErrorMessage = "Enter a valid email address")]
        public string Email { get; set; }
    }
}
```

```

[Required(ErrorMessage = "Mobile number is required")]
[RegularExpression(@"^01[3-9]\d{8}$",
ErrorMessage = "Use a Bangladeshi mobile number, e.g. 01712345678")]
[Display(Name = "Mobile number")]
public string Phone { get; set; }

[Required(ErrorMessage = "School CGPA is required")]
[Range(0.0, 4.0, ErrorMessage = "CGPA must be between 0.00 and 4.00")]
[Display(Name = "School CGPA")]
public double SchoolCgpa { get; set; }

[Required(ErrorMessage = "Choose your date of birth")]
[DataType(DataType.Date)]
[Display(Name = "Date of birth")]
public DateTime DateOfBirth { get; set; }

[Required(ErrorMessage = "Password is required")]
[DataType(DataType.Password)]
[StringLength(30, MinimumLength = 6)]
public string Password { get; set; }

[Required(ErrorMessage = "Please confirm the password")]
[DataType(DataType.Password)]
[Compare("Password", ErrorMessage = "The two passwords do not match")]
[Display(Name = "Confirm password")]
public string ConfirmPassword { get; set; }
}
}

```

Task 2 The Apply Action Pair (25 min)

Step 1 — Create AdmissionController

```

Controllers/AdmissionController.cs

using Microsoft.AspNetCore.Mvc;
using UniPortalMVC.Models;

namespace UniPortalMVC.Controllers
{
    public class AdmissionController : Controller
    {
        [HttpGet]
        public IActionResult Apply()
        {
            return View(new AdmissionApplication());
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Apply(AdmissionApplication form)
        {
            if (!ModelState.IsValid)
            {

```

```

return View(form);
}
TempData["Applicant"] = form.ApplicantName;
return RedirectToAction("Success");
}

public IActionResult Success()
{
    ViewBag.Name = TempData["Applicant"] as string;
    return View();
}
}
}

```

Task 3 The Apply View with Validation Spans (35 min)

Step 1 — Type the form using tag helpers

Views/Admission/Apply.cshtml

```

@model UniPortalMVC.Models.AdmissionApplication
@{ ViewData["Title"] = "Apply for Admission"; }

<h2>Apply for Admission – Spring 2026</h2>

<form asp-action="Apply" method="post" class="col-md-6">
<div asp-validation-summary="ModelOnly" class="text-danger"></div>

<div class="mb-3">
<label asp-for="ApplicantName" class="form-label"></label>
<input asp-for="ApplicantName" class="form-control" />
<span asp-validation-for="ApplicantName" class="text-danger"></span>
</div>

<div class="mb-3">
<label asp-for="Email" class="form-label"></label>
<input asp-for="Email" class="form-control" />
<span asp-validation-for="Email" class="text-danger"></span>
</div>

<div class="mb-3">
<label asp-for="Phone" class="form-label"></label>
<input asp-for="Phone" class="form-control" />
<span asp-validation-for="Phone" class="text-danger"></span>
</div>

<div class="mb-3">
<label asp-for="SchoolCgpa" class="form-label"></label>
<input asp-for="SchoolCgpa" class="form-control" />
<span asp-validation-for="SchoolCgpa" class="text-danger"></span>
</div>

<div class="mb-3">
<label asp-for="DateOfBirth" class="form-label"></label>

```

```

<input asp-for="DateOfBirth" class="form-control" />
<span asp-validation-for="DateOfBirth" class="text-danger"></span>
</div>

<div class="mb-3">
<label asp-for="Password" class="form-label"></label>
<input asp-for="Password" class="form-control" />
<span asp-validation-for="Password" class="text-danger"></span>
</div>

<div class="mb-3">
<label asp-for="ConfirmPassword" class="form-label"></label>
<input asp-for="ConfirmPassword" class="form-control" />
<span asp-validation-for="ConfirmPassword" class="text-danger"></span>
</div>

<button type="submit" class="btn btn-primary">Submit Application</button>
</form>

@section Scripts {
<partial name="_ValidationScriptsPartial" />
}

```

Task 4 Test Client and Server Validation (30 min)

1. Submit an empty form — inline messages appear instantly (client side)
2. Enter Cgpa 4.5 — the Range message appears without a round trip
3. Open DevTools, disable JavaScript, submit again — the page posts, the server re-validates, and the same errors render from ModelState
4. Complete the form correctly — you land on /Admission/Success with your name

Tip: Watch the network tab

With JavaScript enabled, invalid submits produce no POST request at all. With it disabled you will see the POST return the form again with status 200 — server validation doing its job.

Task 5 Other Binding Sources in Action (20 min)

Step 1 — Add a query-bound statistics action

```

Controllers/AdmissionController.cs (add action)
// GET /Admission/Statistics?year=2026
public IActionResult Statistics([FromQuery] int year)
{
    ViewBag.Year = year;
    ViewBag.Applications = 120; // stand-in number for the demo
    return View();
}

```

1. Create a tiny Statistics view showing ViewBag.Year and ViewBag.Applications
2. Visit /Admission/Statistics?year=2026 and confirm the binding

3. Visit /Admission/Statistics and confirm year binds as 0 — default of int

5. Common Errors and Solutions

Error	Likely Cause	Solution
No inline errors appear	_ValidationScriptsPartial not rendered	Add the partial inside @section Scripts of the form view
The required field attribute does nothing	Attribute name typo	Attributes are compiled classes; fix the spelling and rebuild
400 on submit with no message	Antiforgery token missing	Keep [ValidateAntiForgeryToken] and start the form with a form tag helper so the token auto-inserts
Compare always fails	Compared property name wrong	[Compare("Password")] must match the C# property name exactly
Model fields empty after POST	Input names do not match properties	Use asp-for so name/id attributes are generated from the model

6. Completion Checklist

#	Checklist Item	Done
1	AdmissionApplication model typed with six annotation kinds	☐
2	Apply GET/POST pair with ValidateAntiForgeryToken and IsValid guard	☐
3	Form view renders inline errors for every field	☐
4	Client validation confirmed, then server validation confirmed with JS off	☐
5	Successful submit redirects to the Success page	☐
6	Query-bound Statistics action verified with and without the parameter	☐

7. Self-Assessment Questions

- Name four binding sources and give a campus example of each.
- Why is ModelState.IsValid still required when client validation is on?
- What does [Compare] do and on which property must it be placed?
- What is the POST-Redirect-GET pattern protecting against?
- What happens to an unbound int parameter when the query string omits it?

8. Further Reading

- Model binding: <https://learn.microsoft.com/aspnet/core/mvc/models/model-binding>
- Model validation: <https://learn.microsoft.com/aspnet/core/mvc/models/validation>
- Tag helpers for forms: <https://learn.microsoft.com/aspnet/core/mvc/views/working-with-forms>

End of Phase — next: Phase 5 goes beyond the built-in rules — custom validation.

Phase 5 — Custom Validation Attributes

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 5 — Custom Validation Attributes
Syllabus Module	Module 5 — Custom Validation
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual — project-based practice
Project	MediCare — Hospital Management System
Prerequisite	Phase 4 completed — admission form validates with annotations

1. Phase Objectives

By the end of this phase, you will be able to:

- Decide when the built-in annotations are not enough
- Write a ValidationAttribute subclass with a parameterised rule
- Validate one property conditionally on another property
- Validate against a service (the database pattern) via ValidationContext
- Add client-side support for a custom rule with IClientModelValidator

2. Prerequisites

- Phase 4 completed — Apply form renders inline errors
- Theory class Module 5 (Custom Validation) attended

3. Theory Recap

3.1 When Built-Ins Are Not Enough

Data annotations cover shape rules — required, length, range, pattern. Campus rules are often relational or stateful: a preferred enrolment date must lie in the future, a scholarship type matters only when the applicant asked for a scholarship, and an email must not belong to an already registered student. None of these exist as ready-made attributes, so the syllabus teaches three escalation levels: a pure rule attribute, a cross-property attribute, and a service-backed attribute.

3.2 The ValidationAttribute Contract

Every custom rule derives from `ValidationAttribute` and overrides `IsValid(object value, ValidationContext context)`. Return `ValidationResult.Success` to pass, or a `ValidationResult` with an error message to fail. When the rule needs something from the wider application — a `DbContext`, a cache, a registry — `context.GetService(typeof(T))` resolves it from dependency injection.

4. Phase Tasks

Warning: Important

Type all code manually. Custom validation is a favourite quiz topic.

Task 1 FutureDateAttribute — Parameterised Rule (25 min)

Step 1 — Create the Validations folder and attribute

Validations/FutureDateAttribute.cs

```
using System.ComponentModel.DataAnnotations;

namespace UniPortalMVC.Validations
{
    public class FutureDateAttribute : ValidationAttribute
    {
        private readonly int _maxDaysAhead;

        public FutureDateAttribute(int maxDaysAhead = 365)
        {
            _maxDaysAhead = maxDaysAhead;
        }

        protected override ValidationResult IsValid(object value,
            ValidationContext validationContext)
        {
            if (value is not DateTime date)
            {
                return ValidationResult.Success; // [Required] handles empty
            }
        }
    }
}
```

```
if (date.Date <= DateTime.Today)
{
    return new ValidationResult(
        "Enrolment date must be in the future");
}

if (date.Date > DateTime.Today.AddDays(_maxDaysAhead))
{
    return new ValidationResult(
        $"Enrolment cannot be more than {_maxDaysAhead} days ahead");
}

return ValidationResult.Success;
}
}
}
```

Task 2 ConditionalRequiredAttribute — Cross-Property Rule (30 min)

Step 1 — Read the sibling property through the context

Validations/ConditionalRequiredAttribute.cs

```
using System.ComponentModel.DataAnnotations;
using System.Reflection;

namespace UniPortalMVC.Validations
{
    public class ConditionalRequiredAttribute : ValidationAttribute
    {
        private readonly string _dependentProperty;

        public ConditionalRequiredAttribute(string dependentProperty)
        {
            _dependentProperty = dependentProperty;
        }

        protected override ValidationResult IsValid(object value,
            ValidationContext validationContext)
        {
            var flag = validationContext.ObjectType.GetProperty(_dependentProperty)
                ?.GetValue(validationContext.ObjectInstance, null);

            // Rule fires only when the sibling bool property is true
            if (flag is bool on && on)
            {
                if (value is string s && string.IsNullOrEmpty(s))
                {
                    return new ValidationResult(
                        ErrorMessage ?? $"{validationContext.DisplayName} is required "
                        + $"when {_dependentProperty} is ticked");
                }
            }

            return ValidationResult.Success;
        }
    }
}
```

```
}  
}  
}
```

Task 3 UniqueEmailAttribute — Service-Backed Rule (30 min)

Step 1 — Define a registry service (database pattern)

Validations/IApplicantRegistry.cs

```
namespace UniPortalMVC.Validations  
{  
    public interface IApplicantRegistry  
    {  
        bool Exists(string email);  
    }  
  
    public class InMemoryApplicantRegistry : IApplicantRegistry  
    {  
        private readonly List<string> _emails = new()  
        { "tasin.rahman@aiub.edu", "nusrat.jahan@aiub.edu" };  
  
        public bool Exists(string email) =>  
            _emails.Contains(email.Trim().ToLowerInvariant());  
    }  
}
```

Step 2 — Register the service in Program.cs

Program.cs (before builder.Build())

```
using UniPortalMVC.Validations;  
  
builder.Services.AddSingleton<IApplicantRegistry, InMemoryApplicantRegistry>();
```

Step 3 — The attribute resolves the service through the context

Validations/UniqueEmailAttribute.cs

```
using System.ComponentModel.DataAnnotations;  
  
namespace UniPortalMVC.Validations  
{  
    public class UniqueEmailAttribute : ValidationAttribute  
    {  
        protected override ValidationResult IsValid(object value,  
            ValidationContext validationContext)  
        {  
            var registry = (IApplicantRegistry)validationContext  
                .GetService(typeof(IApplicantRegistry));  
  
            if (registry == null)  
            {  
                throw new InvalidOperationException(  
                    "IApplicantRegistry is not registered in Program.cs");  
            }  
        }  
    }  
}
```

```

if (value is string email && registry.Exists(email))
{
    return new ValidationResult(
        $"Email {email} already exists in the UniPortal system");
}
return ValidationResult.Success;
}
}
}

```

Note: Database pattern

Replace `InMemoryApplicantRegistry` with a class that queries a `DbContext` and the attribute code does not change at all — that is why the syllabus calls this the DB validation pattern. You will wire the real database in Phase 7.

Task 4 Apply the Attributes and Extend the Form (25 min)

Step 1 — Update `AdmissionApplication.cs`

Models/AdmissionApplication.cs (new fields)

```

using UniPortalMVC.Validations;

// add these properties to the class
[Required(ErrorMessage = "Choose a preferred enrolment date")]
[DataType(DataType.Date)]
[FutureDate(180)]
[Display(Name = "Preferred enrolment date")]
public DateTime EnrolmentDate { get; set; }

[Display(Name = "I am applying for a scholarship")]
public bool ApplyingForScholarship { get; set; }

[ConditionalRequired("ApplyingForScholarship")]
[StringLength(40)]
[Display(Name = "Scholarship type")]
public string ScholarshipType { get; set; }

```

Step 2 — Add the inputs to `Apply.cshtml`

Views/Admission/Apply.cshtml (new fields)

```

<div class="form-check mb-3">
    <input asp-for="ApplyingForScholarship" class="form-check-input" />
    <label asp-for="ApplyingForScholarship" class="form-check-label"></label>
</div>

<div class="mb-3">
    <label asp-for="ScholarshipType" class="form-label"></label>
    <input asp-for="ScholarshipType" class="form-control" />
    <span asp-validation-for="ScholarshipType" class="text-danger"></span>
</div>

```

```
<div class="mb-3">
<label asp-for="EnrolmentDate" class="form-label"></label>
<input asp-for="EnrolmentDate" class="form-control" />
<span asp-validation-for="EnrolmentDate" class="text-danger"></span>
</div>
```

Task 5 Add Client Support with IClientModelValidator (25 min)

Step 1 — Implement the interface on FutureDateAttribute

FutureDateAttribute.cs (add method + usings)

```
public IEnumerable<ModelClientValidationRule> GetClientValidationRules(
    ModelMetadata metadata, ClientModelValidationContext context)
{
    var rule = new ModelClientValidationRule(
        "futuredate", ErrorMessage ?? "Date must be in the future");
    rule.ValidationParameters["maxdays"] = _maxDaysAhead;
    yield return rule;
}
```

Step 2 — Teach jQuery unobtrusive the new rule

wwwroot/js/site.js (append)

```
// wwwroot/js/site.js (append)
$.validator.addMethod("futuredate", function (value, element, params) {
    if (!value) return true;
    var d = new Date(value);
    var today = new Date(); today.setHours(0, 0, 0, 0);
    var max = new Date();
    max.setDate(today.getDate() + params.maxdays);
    return d > today && d <= max;
});

$.validator.unobtrusive.adapters.add(
    "futuredate", ["maxdays"],
    function (options) {
        options.rules["futuredate"] = {
            maxdays: options.params.maxdays
        };
        options.messages["futuredate"] = options.message;
    });
```

1. Rebuild and hard-refresh the page (Ctrl+F5)
2. Pick yesterday — the message appears instantly, no round trip

Task 6 Run the Full Validation Matrix (20 min)

Input	Expected Result
Enrolment date = today	Enrolment date must be in the future

Input	Expected Result
Enrolment date = today + 200 days	Enrolment cannot be more than 180 days ahead
Scholarship ticked, type empty	Scholarship type is required message
Scholarship unticked, type empty	Form passes this rule
Email tasin.rahman@aiub.edu	Already exists in the UniPortal system
Email new.student@aiub.edu	Passes uniqueness

5. Common Errors and Solutions

Error	Likely Cause	Solution
Custom rule never runs	Attribute not applied or wrong namespace	Add the using for UniPortalMVC.Validations on the model
GetService returns null	Service not registered in Program.cs	Add AddSingleton()
NullReference in ConditionalRequired	Sibling property name typo	The string in [ConditionalRequired("...")] must match a property name
Client rule ignored	Adapter name mismatch	The unobtrusive adapter name must equal the rule name: futuredate
Site.js changes not loading	Browser cache	Hard refresh with Ctrl+F5 or bump the script version query string

6. Completion Checklist

#	Checklist Item	Done
1	FutureDateAttribute rejects today and dates beyond 180 days	☐
2	ConditionalRequired fires only when the scholarship checkbox is ticked	☐
3	UniqueEmail blocks both seeded emails with the service message	☐
4	IClientModelValidator + site.js adapter produce instant client errors	☐
5	All six matrix rows tested with expected outcomes	☐

7. Self-Assessment Questions

- Why return ValidationResult.Success for a null value inside a custom attribute that also has [Required]?
- When does ValidationContext.GetService become necessary instead of a simple rule?
- How does ConditionalRequired read the sibling property value?
- What two pieces must match for the unobtrusive adapter to fire?

- Why is the registry injected as a singleton here instead of scoped?

8. Further Reading

- Custom validation attributes:
<https://learn.microsoft.com/aspnet/core/mvc/models/validation#custom-attributes>
- IClientModelValidator reference:
<https://learn.microsoft.com/aspnet/core/mvc/models/validation#iclientmodelvalidator>

End of Phase — next: Phase 6 connects the portal to a real database — Database-First.

Phase 6 — Database-First EF Core: UniversityDB

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 6 — Database-First EF Core: UniversityDB
Syllabus Module	Module 6 — Database-First EF Core
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual — project-based practice
Project	MediCare — Hospital Management System
Prerequisite	Phase 5 completed — custom validation working on the admission form

1. Phase Objectives

By the end of this phase, you will be able to:

- Explain when Database-First is the right approach
- Install the EF Core SQL Server packages and the dotnet-ef tool
- Create UniversityDB in SSMS with a Department table and sample rows
- Scaffold the DbContext and entity classes with dotnet ef dbcontext scaffold
- Register the context in DI and build a full Department CRUD

2. Prerequisites

- Phase 5 completed — UniPortalMVC builds and runs
- SQL Server and SSMS installed (Phase 0)
- Theory class Module 6 (Database-First EF Core) attended

3. Theory Recap

3.1 What Is Database-First

In Database-First the database already exists — created by a DBA, an older system, or another team. EF Core reverse-engineers the schema and generates entity classes plus a DbContext for you. Universities are full of such legacy databases: the department registry existed decades before this portal. You will never hand-edit the generated files; regeneration is always done with the scaffold command using the `--force` flag.

3.2 The Scaffold Command

Run from the UniPortalMVC folder

```
dotnet ef dbcontext scaffold "Name=UniversityDb" Microsoft.EntityFrameworkCore.SqlServer
--context-dir EF --output-dir EF/Tables
```

Fragment	Meaning
"Name=UniversityDb"	Reads the connection string named UniversityDb from appsettings.json — the password never lands in source control
--context-dir EF	Folder for the generated UniversityDbContext.cs
--output-dir EF/Tables	Folder for the generated entity classes
--force (optional)	Overwrite previously generated files after the database changes

4. Phase Tasks

Warning: Important

Type all commands manually. The scaffold output must be generated by the tool, never retyped — but the command itself you must type.

Task 1 Create UniversityDB in SSMS (25 min)

Step 1 — Create the database and table visually

1. Open SSMS and connect to your SQL Server instance
2. Right-click Databases, New Database, name it UniversityDB, OK
3. Expand UniversityDB, right-click Tables, New Table
4. Design the columns below, set DepartmentId as PK with Identity(1,1)
5. Save the table as Department

Column	Type	Allow Nulls	Notes
DepartmentId	int	No	Primary key, identity
DepartmentName	nvarchar(60)	No	e.g. Computer Science

Column	Type	Allow Nulls	Notes
OfficeRoom	nvarchar(20)	Yes	e.g. Building B, Room 402
EstablishedDate	date	Yes	Founding year of the department

Step 2 — Insert three sample rows

New Query window in SSMS

```
INSERT INTO Department (DepartmentName, OfficeRoom, EstablishedDate) VALUES
('Computer Science', 'B-402', '1994-01-01'),
('Electrical Engineering', 'C-201', '1995-01-01'),
('Business Administration', 'A-105', '1996-01-01');
```

Task 2 Install EF Core Packages and Tools (20 min)

1. In Visual Studio open Tools, NuGet Package Manager, **Package Manager Console (PMC)**
2. Ensure Default project is UniPortalMVC, then run the two installs below

PMC

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Design
```

Then install (or update) the cross-project CLI tool once per machine:

Developer PowerShell

```
dotnet tool install --global dotnet-ef
dotnet ef --version
```

Warning: Version match

The package versions and the dotnet-ef major version should both be 8.x. A mismatch surfaces later as "unable to create a DbContext" during scaffold.

Task 3 Connection String and Scaffold (25 min)

Step 1 — Add the named connection string

appsettings.json (add ConnectionStrings block)

```
{
  "ConnectionStrings": {
    "UniversityDb": "Server=localhost\\SQLEXPRESS;Database=UniversityDB;Trusted_Connection=True;TrustServerCertificate=True;"
  },
  "Logging": { "LogLevel": { "Default": "Information" } },
  "AllowedHosts": "*"
}
```

Step 2 — Run the scaffold

1. Open Developer PowerShell and cd into C:\AIUB\CSC4164\UniPortal\UniPortalMVC

2. Run the scaffold command from Theory Recap 3.2
3. Confirm EF/UniversityDbContext.cs and EF/Tables/Department.cs appear

EF/Tables/Department.cs (generated, approx.)

```
public partial class Department
{
    public int DepartmentId { get; set; }

    public string DepartmentName { get; set; } = null!;

    public string? OfficeRoom { get; set; }

    public DateTime? EstablishedDate { get; set; }
}
```

Note: Read the generated context

OnConfiguring contains a hardcoded connection string warning comment — you will remove that when registering the context with DI in Task 4. OnModelCreating carries the table and column mappings.

Task 4 Register the Context and Build Department CRUD (35 min)

Step 1 — Register in Program.cs

Program.cs

```
using Microsoft.EntityFrameworkCore;

builder.Services.AddDbContext<UniversityDbContext>(options =>
    options.UseSqlServer(builder.Configuration
        .GetConnectionString("UniversityDb")));
```

Step 2 — Create DepartmentController

Controllers/DepartmentController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using UniPortalMVC.EF;

namespace UniPortalMVC.Controllers
{
    public class DepartmentController : Controller
    {
        private readonly UniversityDbContext _db;

        public DepartmentController(UniversityDbContext db)
        {
            _db = db;
        }

        public IActionResult Index()
        {
            var list = _db.Departments.OrderBy(d => d.DepartmentName).ToList();
        }
    }
}
```

```
return View(list);
}

public IActionResult Details(int id)
{
    var dept = _db.Departments.Find(id);
    if (dept == null) return NotFound();
    return View(dept);
}

[HttpGet]
public IActionResult Create()
{
    return View();
}

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(Department dept)
{
    if (!ModelState.IsValid) return View(dept);
    _db.Departments.Add(dept);
    _db.SaveChanges();
    return RedirectToAction("Index");
}

[HttpGet]
public IActionResult Edit(int id)
{
    var dept = _db.Departments.Find(id);
    if (dept == null) return NotFound();
    return View(dept);
}

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Edit(Department dept)
{
    if (!ModelState.IsValid) return View(dept);
    _db.Departments.Update(dept);
    _db.SaveChanges();
    return RedirectToAction("Index");
}

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Delete(int id)
{
    var dept = _db.Departments.Find(id);
    if (dept != null)
    {
        _db.Departments.Remove(dept);
        _db.SaveChanges();
    }
}
```

```
return RedirectToAction("Index");
}
}
}
```

Task 5 Views and the CRUD Smoke Test (30 min)

Step 1 — Create the views

1. Create Views/Department/Index.cshtml — strongly-typed List table with Edit and Delete buttons per row
2. Create Details.cshtml, Create.cshtml and Edit.cshtml — reuse the form pattern with tag helpers and validation spans from Phase 4
3. Add a Departments link to the layout navbar

Step 2 — Smoke test every verb

1. /Department shows the three seeded rows from SSMS
2. Create adds Mathematics — it appears in the list and in SSMS after refresh
3. Edit renames B-402 to B-403 — change visible in both places
4. Delete removes Mathematics — row gone in both places

5. Common Errors and Solutions

Error	Likely Cause	Solution
dotnet ef: command not found	Tool not installed	dotnet tool install --global dotnet-ef, then reopen the shell
Unable to create a DbContext	Missing Design package or version skew	Install Microsoft.EntityFrameworkCore.Design 8.x and retry
Login failed / certificate error	Connection string wrong	Match the SSMS server name; add TrustServerCertificate=True
scaffold created an OnConfiguring warning	Generated default	Register with AddDbContext in DI and ignore the generated method
Scaffold did not pick up a new column	Files already exist	Re-run with --force after saving the table design in SSMS

6. Completion Checklist

#	Checklist Item	Done
1	UniversityDB.Department created in SSMS with three rows	☐
2	EF Core SqlServer and Design packages installed	☐
3	Scaffold generated UniversityDbContext and Department entity	☐

#	Checklist Item	Done
4	Context registered with AddDbContext and the named connection string	☐
5	Department Index, Details, Create, Edit, Delete all verified	☐
6	One SSMS round trip confirmed data truly persisted	☐

7. Self-Assessment Questions

- Give two situations where Database-First beats Code-First.
- Why should the connection string come from appsettings.json instead of the scaffold default?
- What does the --force flag risk, and why are partial classes the answer?
- Which CRUD method pair belongs to Create: Add or Update?
- Where does EF Core look for the "UniversityDb" name at runtime?

8. Further Reading

- Reverse engineering (scaffolding): <https://learn.microsoft.com/ef/core/managing-schemas/scaffolding>
- EF Core tools reference: <https://learn.microsoft.com/ef/core/cli/dotnet>

End of Phase — next: Phase 7 flips the direction — Code-First with migrations.

Phase 7 — Code-First EF Core: Courses and Enrollments

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 7 — Code-First EF Core: Courses and Enrollments
Syllabus Module	Module 7 — Code-First EF Core
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual — project-based practice
Project	MediCare — Hospital Management System
Prerequisite	Phase 6 completed — Database-First CRUD working against UniversityDB

1. Phase Objectives

By the end of this phase, you will be able to:

- Write entity classes and a DbContext from scratch
- Create and apply migrations, and read Up()/Down()
- Extend the model with a second migration
- Configure one-to-many and many-to-many relationships
- Seed data with HasData and build a Course CRUD

2. Prerequisites

- Phase 6 completed — EF Core packages and dotnet-ef already installed
- Theory class Module 7 (Code-First EF Core) attended

3. Theory Recap

3.1 Code-First vs Database-First

Aspect	Database-First (Phase 6)	Code-First (today)
Source of truth	The database	Your C# classes
Classes come from	Scaffolding	You type them
Schema changes	Edit in SSMS, scaffold --force	Edit classes, add a migration
Version control of schema	Hard	Migration files in Git

3.2 The Migration Workflow

```
Developer PowerShell (inside UniPortalMVC folder)
dotnet ef migrations add InitialCreate -c UniPortalDbContext
dotnet ef database update -c UniPortalDbContext
```

Each migration is a C# class with Up() applying the change and Down() reverting it. The database update plays the pending Up() methods in order. A second migration (AddGradeToEnrollment) later in this phase shows how the schema evolves without touching SSMS at all.

4. Phase Tasks

Warning: Important

Type all code manually. This database belongs to your code forever.

Task 1 Entity Classes for the Course Domain (30 min)

Step 1 — Create the three entities under Models/CourseDb

Models/CourseDb/Course.cs (all three classes)

```
using System.ComponentModel.DataAnnotations;

namespace UniPortalMVC.Models.CourseDb
{
    public class Course
    {
        [Key]
        public int CourseId { get; set; }

        [Required, StringLength(10)]
        public string CourseCode { get; set; }

        [Required, StringLength(80)]
        public string Title { get; set; }

        [Range(1, 6)]
        public int CreditHours { get; set; }

        public List<Enrollment> Enrollments { get; set; } = new();
    }

    public class Student
    {
        [Key]
        public int StudentId { get; set; }

        [Required, StringLength(60)]
        public string FullName { get; set; }

        [Required, StringLength(80)]
        public string Email { get; set; }

        [StringLength(15)]
        public string MatricId { get; set; }

        public List<Enrollment> Enrollments { get; set; } = new();
    }

    public class Enrollment
    {
        [Key]
        public int EnrollmentId { get; set; }

        public int StudentId { get; set; }
        public Student Student { get; set; }

        public int CourseId { get; set; }
        public Course Course { get; set; }
    }
}
```

```
[Required, StringLength(15)]
public string Semester { get; set; }
}
}
```

Note: Many-to-many with a payload

Student and Course are linked by Enrollment, which carries its own data (Semester, and Grade coming in Task 4). EF Core calls this an explicit join entity — exactly the pattern in your syllabus.

Task 2 The UniPortalDbContext (20 min)

Step 1 — Create the context class

EF/UniPortalDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using UniPortalMVC.Models.CourseDb;

namespace UniPortalMVC.EF
{
    public class UniPortalDbContext : DbContext
    {
        public UniPortalDbContext(DbContextOptions<UniPortalDbContext> options)
            : base(options) { }

        public DbSet<Course> Courses { get; set; }
        public DbSet<Student> Students { get; set; }
        public DbSet<Enrollment> Enrollments { get; set; }

        protected override void OnModelCreating(ModelBuilder mb)
        {
            mb.Entity<Enrollment>(e =>
            {
                e.HasOne(x => x.Student)
                  .WithMany(s => s.Enrollments)
                  .HasForeignKey(x => x.StudentId)
                  .OnDelete(DeleteBehavior.Cascade);

                e.HasOne(x => x.Course)
                  .WithMany(c => c.Enrollments)
                  .HasForeignKey(x => x.CourseId)
                  .OnDelete(DeleteBehavior.Cascade);
            });
        }
    }
}
```

Step 2 — Register it and add the connection string

Program.cs

```
builder.Services.AddDbContext<UniPortalDbContext>(options =>
options.UseSqlServer(builder.Configuration
```

```
.GetConnectionString("UniPortalDb")));
```

appsettings.json (second connection string)

```
"UniPortalDb": "Server=localhost\\SQLEXPRESS;Database=UniPortalDb;Trusted_Connection=True;TrustServerCertificate=True;"
```

Task 3 Initial Migration and Database Update (20 min)

1. From Developer PowerShell inside the UniPortalMVC folder, add the InitialCreate migration and update the database
2. Open SSMS and confirm the UniPortalDb database exists with the three tables and correct foreign keys
3. Open the generated Migration file and read Up(): three CreateTable calls plus Foreign keys, and Down(): the drop calls

Tip: Two contexts, one project

Always pass -c UniPortalDbContext so the tool targets the right context — UniversityDbContext from Phase 6 also lives in this project.

Task 4 Second Migration — Add Grade to Enrollment (20 min)

Step 1 — Extend the entity, then migrate

Models/CourseDb/Course.cs (Enrollment change)

```
[StringLength(2)]
public string? Grade { get; set; } // add inside Enrollment
```

Developer PowerShell

```
dotnet ef migrations add AddGradeToEnrollment -c UniPortalDbContext
dotnet ef database update -c UniPortalDbContext
```

1. Read the new migration: Up() calls AddColumn, Down() calls DropColumn
2. Confirm the Grade column exists in SSMS

Task 5 Seed Data with HasData (25 min)

Step 1 — Add seed rows in OnModelCreating

EF/UniPortalDbContext.cs (OnModelCreating additions)

```
mb.Entity<Course>().HasData(
    new Course { CourseId = 1, CourseCode = "CSC4164",
    Title = "Advanced Programming", CreditHours = 3 },
    new Course { CourseId = 2, CourseCode = "MAT2101",
    Title = "Linear Algebra", CreditHours = 3 },
    new Course { CourseId = 3, CourseCode = "PHY1102",
    Title = "Physics II", CreditHours = 3 });

mb.Entity<Student>().HasData(
    new Student { StudentId = 1, FullName = "Tasin Rahman",
    Email = "tasin.rahman@aiub.edu", MatricId = "23-12345-1" },
    new Student { StudentId = 2, FullName = "Nusrat Jahan",
```

```
Email = "nusrat.jahan@aiub.edu", MatricId = "23-12346-1" });

mb.Entity<Enrollment>().HasData(
    new Enrollment { EnrollmentId = 1, StudentId = 1, CourseId = 1,
        Semester = "Spring2026", Grade = null },
    new Enrollment { EnrollmentId = 2, StudentId = 1, CourseId = 2,
        Semester = "Spring2026", Grade = null });
```

Developer PowerShell

```
dotnet ef migrations add SeedCampusData -c UniPortalDbContext
dotnet ef database update -c UniPortalDbContext
```

Warning: Explicit keys required

HasData rows must set primary keys explicitly, and EF inserts them on the very first migration that contains them — never edit seeded rows by hand afterwards; add a new migration instead.

Task 6 CourseController CRUD (30 min)

Controllers/CourseController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using UniPortalMVC.EF;
using UniPortalMVC.Models.CourseDb;

namespace UniPortalMVC.Controllers
{
    public class CourseController : Controller
    {
        private readonly UniPortalDbContext _db;

        public CourseController(UniPortalDbContext db)
        {
            _db = db;
        }

        public IActionResult Index()
        {
            var courses = _db.Courses
                .Include(c => c.Enrollments)
                .OrderBy(c => c.CourseCode)
                .ToList();
            return View(courses);
        }

        [HttpGet]
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
```

```

public IActionResult Create(Course course)
{
    if (!ModelState.IsValid) return View(course);
    _db.Courses.Add(course);
    _db.SaveChanges();
    return RedirectToAction("Index");
}

[HttpGet]
public IActionResult Edit(int id)
{
    var course = _db.Courses.Find(id);
    if (course == null) return NotFound();
    return View(course);
}

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Edit(Course course)
{
    if (!ModelState.IsValid) return View(course);
    _db.Courses.Update(course);
    _db.SaveChanges();
    return RedirectToAction("Index");
}

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Delete(int id)
{
    var course = _db.Courses.Find(id);
    if (course != null)
    {
        _db.Courses.Remove(course);
        _db.SaveChanges();
    }
    return RedirectToAction("Index");
}
}
}
}

```

1. Create the four views following the Department pattern from Phase 6
2. CourseCode CSC4164 appears with 1 enrolled student via Include
3. Add a new course CSC1101 — visible in SSMS without touching the DB

5. Common Errors and Solutions

Error	Likely Cause	Solution
Unable to resolve service for UniPortalDbContext	Missing registration	Add the AddDbContext block before builder.Build()

Error	Likely Cause	Solution
Cannot scaffold two contexts ambiguously	Two contexts in project	Always pass -c to migrations and update
The INSERT statement conflicted with the FOREIGN KEY	Seed order	Seed parents (Course, Student) before Enrollments
SqlException: Column Grade does not exist	Migration not applied	Run dotnet ef database update after adding the migration
Model changes ignored	Forgot the second migration	Code-First requires migrations add AND database update

6. Completion Checklist

#	Checklist Item	Done
1	Course, Student, Enrollment entities typed with data annotations	☐
2	UniPortalDbContext registered with its own connection string	☐
3	InitialCreate migration applied — tables visible in SSMS	☐
4	AddGradeToEnrollment migration applied — Grade column visible	☐
5	Seed rows inserted via HasData and confirmed in SSMS	☐
6	Course CRUD works end to end with Include showing enrollment counts	☐

7. Self-Assessment Questions

- What do Up() and Down() do in a migration class?
- Why does an explicit join entity beat a bare many-to-many here?
- Why must HasData rows contain explicit primary keys?
- What happens if you edit entity classes but never add a migration?
- How do two DbContexts coexist in one MVC project?

8. Further Reading

- Migrations overview: <https://learn.microsoft.com/ef/core/managing-schemas/migrations>
- Creating and configuring a model: <https://learn.microsoft.com/ef/core/modeling>
- Data seeding: <https://learn.microsoft.com/ef/core/modeling/data-seeding>

End of Phase — next: Phase 8 adds state — session cart and cookies.

Phase 8 — Session and Cookies: the Registration Cart

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 8 — Session and Cookies: the Registration Cart
Syllabus Module	Module 8 — Session and Cookie Management
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual — project-based practice
Project	MediCare — Hospital Management System
Prerequisite	Phase 7 completed — courses and enrollments live in UniPortalDb

1. Phase Objectives

By the end of this phase, you will be able to:

- Decide between session state and cookies using the comparison table
- Enable session with the right options and middleware order
- Build a course registration cart stored in session
- Read and write cookies with CookieOptions
- Apply the security checklist: HttpOnly, Secure, expiration, validation

2. Prerequisites

- Phase 7 completed — Course CRUD works with seeded data
- Theory class Module 8 (Session and Cookies) attended

3. Theory Recap

3.1 Session vs Cookies

Aspect	Session	Cookie
Stored	Server side	Browser, sent with every request
Capacity	Large (memory bound)	About 4 KB per cookie

Aspect	Session	Cookie
Security	Never leaves the server	Visible to the client; sign/encrypt or keep values harmless
Lifetime	IdleTimeout (you set 30 min)	Expiry you choose in CookieOptions
UniPortal use	Registration cart	Theme, last course, remembered id

3.2 Enabling Session

Program.cs (session wiring, order matters)

```
builder.Services.AddDistributedMemoryCache(); // backing store
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});

var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.UseSession(); // BEFORE MapControllerRoute
app.MapControllerRoute(...);
```

4. Phase Tasks

Warning: Important

Type all code manually. Middleware order bugs are silent and famous.

Task 1 Wire Session into the Pipeline (15 min)

Step 1 — Edit Program.cs exactly as in Theory Recap 3.2

1. Add the three service lines before builder.Build()
2. Add app.UseSession() between UseRouting and MapControllerRoute
3. Run the app once — nothing visible changes, but HttpContext.Session now works everywhere

Warning: Order check

If UseSession runs after MapControllerRoute, actions that touch Session throw InvalidOperationException: Session has not been configured.

Task 2 The Registration Cart in Session (35 min)

Step 1 — Create RegistrationController

Controllers/RegistrationController.cs

```
using Microsoft.AspNetCore.Mvc;
using UniPortalMVC.EF;

namespace UniPortalMVC.Controllers
{
    public class RegistrationController : Controller
    {
        private readonly UniPortalDbContext _db;

        public RegistrationController(UniPortalDbContext db)
        {
            _db = db;
        }

        // GET /Registration/Add/2
        public IActionResult Add(int id)
        {
            var course = _db.Courses.Find(id);
            if (course == null) return NotFound();

            var cart = HttpContext.Session.GetString("Cart") ?? "";
            var ids = cart.Split(',', StringSplitOptions.RemoveEmptyEntries)
                .ToList();
            if (!ids.Contains(id.ToString()))
            {
                ids.Add(id.ToString());
                HttpContext.Session.SetString("Cart", string.Join(",", ids));
            }
            TempData["Message"] = $"{course.CourseCode} added to your cart";
            return RedirectToAction("Cart");
        }

        public IActionResult Cart()
        {
            var cart = HttpContext.Session.GetString("Cart") ?? "";
            var ids = cart.Split(',', StringSplitOptions.RemoveEmptyEntries)
                .Select(int.Parse).ToList();
            var courses = _db.Courses
                .Where(c => ids.Contains(c.CourseId)).ToList();
            ViewBag.TotalCredits = courses.Sum(c => c.CreditHours);
            return View(courses);
        }

        // GET /Registration/Remove/2
        public IActionResult Remove(int id)
        {
            var cart = HttpContext.Session.GetString("Cart") ?? "";
            var ids = cart.Split(',', StringSplitOptions.RemoveEmptyEntries)
                .Where(x => x != id.ToString()).ToList();
            HttpContext.Session.SetString("Cart", string.Join(",", ids));
            return RedirectToAction("Cart");
        }

        public IActionResult Clear()
```

```
{
    HttpContext.Session.Remove("Cart");
    return RedirectToAction("Cart");
}
}
```

Step 2 — Create Views/Registration/Cart.cshtml

Views/Registration/Cart.cshtml

```
@model List<UniPortalMVC.Models.CourseDb.Course>
@{ ViewData["Title"] = "Registration Cart"; }

<h2>Course Registration Cart</h2>

@if (TempData["Message"] != null)
{
    <div class="alert alert-success">@TempData["Message"]</div>
}

@if (Model.Count == 0)
{
    <p>Your cart is empty. <a asp-controller="course" asp-action="Index">
    Browse courses</a></p>
}
else
{
    <table class="table">
    <tr><th>Code</th><th>Title</th><th>Credits</th><th></th></tr>
    @foreach (var c in Model)
    {
        <tr>
        <td>@c.CourseCode</td>
        <td>@c.Title</td>
        <td>@c.CreditHours</td>
        <td><a asp-controller="registration" asp-action="Remove"
        asp-route-id="@c.CourseId" class="text-danger">Remove</a></td>
        </tr>
    }
    <tr><th colspan="2">Total</th><th>@ViewBag.TotalCredits</th><th></th></tr>
    </table>
}
```

1. Add an "Add to cart" link on the Course Index rows
2. Add courses 1 and 2 to the cart, confirm total credits
3. Remove one, then Clear — the empty message returns

Task 3 Cookies: Theme and Last Visited Course (30 min)

Step 1 — Write the theme cookie

RegistrationController.cs (add action)

```
public IActionResult SetTheme(string theme)
```

```
{
    var allowed = new[] { "light", "dark" };
    if (!allowed.Contains(theme))
    {
        return BadRequest("Unknown theme");
    }

    Response.Cookies.Append("CampusTheme", theme, new CookieOptions
    {
        Expires = DateTimeOffset.UtcNow.AddDays(30),
        HttpOnly = true,
        Secure = true,
        IsEssential = false
    });
    return RedirectToAction("Index", "Home");
}
```

Step 2 — Read the cookie in the layout

Views/Shared/_Layout.cshtml (top of file)

```
@{
    var theme = Context.Request.Cookies["CampusTheme"] ?? "light";
}
<html data-theme="@theme">
```

Step 3 — Last visited course cookie (short-lived)

Controllers/CourseController.cs (Details addition)

```
// inside CourseController.Details, before return View(course)
Response.Cookies.Append("LastCourse", course.CourseCode,
    new CookieOptions { Expires = DateTimeOffset.UtcNow.AddMinutes(30) });
```

1. Visit /Registration/SetTheme?theme=dark, then view source — data-theme="dark"
2. Open /students/2, then check DevTools, Application, Cookies — LastCourse=CSC4164 with 30-minute expiry

Task 4 Remember My Student Number (Persistent Cookie) (20 min)

Step 1 — Checkbox on the Cart page

Views/Registration/Cart.cshtml (form addition)

```
<form asp-controller="registration" asp-action="Remember" method="post"
class="d-flex gap-2 align-items-center mb-3">
<input type="text" name="studentNumber" class="form-control w-auto"
placeholder="23-12345-1"
value="@Context.Request.Cookies["RememberStudent"]" />
<button class="btn btn-outline-primary btn-sm">Remember me</button>
</form>
```

RegistrationController.cs (Remember action)

```
[HttpPost]
[ValidateAntiForgeryToken]
```

```

public IActionResult Remember(string studentNumber)
{
    if (string.IsNullOrEmpty(studentNumber))
    {
        Response.Cookies.Delete("RememberStudent");
    }
    else
    {
        Response.Cookies.Append("RememberStudent", studentNumber,
            new CookieOptions
            {
                Expires = DateTimeOffset.UtcNow.AddDays(14),
                HttpOnly = true
            });
    }
    return RedirectToAction("Cart");
}

```

1. Submit 23-12345-1, close the browser, reopen /Registration/Cart — the input is pre-filled from the cookie
2. Submit empty — the cookie is deleted and the field is blank

Task 5 Security Pass (15 min)

Control	Where You Used It
HttpOnly	Session cookie + CampusTheme + RememberStudent — JS cannot read them
Secure	CampusTheme — travels only over HTTPS in production
Expiration	30 days for theme, 30 minutes for LastCourse, 14 days for RememberStudent
Allow-list validation	SetTheme rejects values outside light/dark
Session IsEssential	Cart keeps working when consent tracking is off

5. Common Errors and Solutions

Error	Likely Cause	Solution
Session has not been configured	UseSession missing or misplaced	Place app.UseSession() before MapControllerRoute
Cart empties on restart	Memory cache is per-process	Expected with AddDistributedMemoryCache; use a distributed cache in production
Cookie never persists	No Expires in CookieOptions	A cookie without Expires dies with the browser session
Theme flashes light on load	Cookie read after HTML render	Read Context.Request.Cookies early in the layout top-level code

Error	Likely Cause	Solution
Clearing session throws	Session not enabled for that path	Keep IsEssential = true so GDPR consent does not kill the cart

6. Completion Checklist

#	Checklist Item	Done
1	Session configured with IdleTimeout 30, HttpOnly, IsEssential	☐
2	UseSession placed correctly in the middleware order	☐
3	Cart add, view, remove and clear all verified with credit totals	☐
4	CampusTheme cookie survives 30 days and dark mode renders	☐
5	LastCourse cookie written on Details with 30-minute expiry	☐
6	RememberStudent prefills after a full browser restart	☐

7. Self-Assessment Questions

- Give one piece of data that must live in session and one that must not live in a cookie.
- Why does UseSession need to run before MapControllerRoute?
- What happens to a session cookie without Expires when the browser closes?
- Why validate the theme value before writing it to a cookie?
- Why did the cart survive across actions without a database write?

8. Further Reading

- Session in ASP.NET Core: <https://learn.microsoft.com/aspnet/core/fundamentals/app-state>
- Cookies and consent: <https://learn.microsoft.com/aspnet/core/security/gdpr>

End of Phase — next: Phase 9 cleans the data layer — DTOs and AutoMapper.

Phase 9 — DTOs and AutoMapper

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 9 — DTOs and AutoMapper
Syllabus Module	Module 9 — DTOs and AutoMapper
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual — project-based practice
Project	MediCare — Hospital Management System
Prerequisite	Phase 8 completed — cart works in session; enrollments in UniPortalDb

1. Phase Objectives

By the end of this phase, you will be able to:

- Explain why entities must not leak into views or APIs
- Design Create, Update and Read DTOs per operation
- Install AutoMapper and register a MappingProfile
- Flatten computed members with ForMember
- Refactor the student admin screens onto DTOs

2. Prerequisites

- Phase 7 completed — Student entity with Enrollments in UniPortalDb
- Theory class Module 9 (DTOs and AutoMapper) attended

3. Theory Recap

3.1 Entity vs DTO

Concern	Entity (Student)	DTO (StudentReadDTO)
Purpose	Maps a table row	Shapes data for one screen or API call
Exposes	Every column	Only what the consumer needs
Risk	Over-posting, leaked fields	None — you choose the fields
Lifetime	Tied to the schema	Evolves per operation

3.2 AutoMapper Wiring

AutoMapper moves data between entity and DTO so you never hand-copy properties. One MappingProfile class declares every map; DI discovers it at startup with AddAutoMapper, and controllers inject IMapper and call Map. The registration and reading of maps happen once per application, not per request.

```
PMC (AutoMapper.Extensions.Microsoft.DependencyInjection where required by your .NET version)
```

```
Install-Package AutoMapper
```

4. Phase Tasks

Warning: Important

Type all code manually. Mapping profiles fail at startup, not at build — run after every change.

Task 1 Design the Three DTOs (25 min)

Step 1 — Create the DTOs folder and classes

```
DTOs/StudentDTOs.cs
```

```
using System.ComponentModel.DataAnnotations;

namespace UniPortalMVC.DTOs
{
    public class StudentCreatedDTO
    {
        [Required, StringLength(60, MinimumLength = 3)]
        public string FullName { get; set; }

        [Required, EmailAddress]
        public string Email { get; set; }

        [Required, StringLength(15)]
        public string MatricId { get; set; }
    }

    public class StudentUpdatedDTO
    {
        public int StudentId { get; set; }

        [Required, StringLength(60, MinimumLength = 3)]
        public string FullName { get; set; }

        [StringLength(15)]
        public string MatricId { get; set; }
    }

    public class StudentReadDTO
    {
        public int StudentId { get; set; }
    }
}
```

```
public string FullName { get; set; }
public string Email { get; set; }
public string MatricId { get; set; }
public int CourseCount { get; set; }
}
}
```

Note: One DTO per operation

Create has no id and no CourseCount. Update cannot change the email (the stable identifier). Read shows the enrollment count — data no create form should carry. The syllabus calls this per-operation DTOs.

Task 2 MappingProfile with ForMember (25 min)**Step 1 — Create Profiles/MappingProfile.cs**

```
Profiles/MappingProfile.cs

using AutoMapper;
using UniPortalMVC.DTOs;
using UniPortalMVC.Models.CourseDb;

namespace UniPortalMVC.Profiles
{
    public class MappingProfile : Profile
    {
        public MappingProfile()
        {
            CreateMap<StudentCreatedTO, Student>();
            CreateMap<StudentUpdatedTO, Student>()
                .ForMember(d => d.Email, o => o.Ignore());

            CreateMap<Student, StudentReadDTO>()
                .ForMember(d => d.CourseCount,
                    o => o.MapFrom(s => s.Enrollments.Count));

            CreateMap<StudentCreatedTO, StudentUpdatedTO>()
                .ReverseMap();
        }
    }
}
```

Step 2 — Register AutoMapper

```
Program.cs

using UniPortalMVC.Profiles;

builder.Services.AddAutoMapper(typeof(MappingProfile));
```

Tip: Flattening in action

CourseCount does not exist on Student. ForMember computes it from the Enrollments collection — the classic "flatten" pattern from your syllabus, and the reason Include(Enrollments) matters when you query.

Task 3 Refactor the Student Admin Flow (35 min)

Step 1 — Create StudentAdminController

Controllers/StudentAdminController.cs

```
using AutoMapper;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using UniPortalMVC.DTOS;
using UniPortalMVC.EF;
using UniPortalMVC.Models.CourseDb;

namespace UniPortalMVC.Controllers
{
    public class StudentAdminController : Controller
    {
        private readonly UniPortalDbContext _db;
        private readonly IMapper _mapper;

        public StudentAdminController(UniPortalDbContext db, IMapper mapper)
        {
            _db = db;
            _mapper = mapper;
        }

        public IActionResult Index()
        {
            var students = _db.Students
                .Include(s => s.Enrollments)
                .ToList();
            var dto = _mapper.Map<List<StudentReadDTO>>(students);
            return View(dto);
        }

        [HttpGet]
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Create(StudentCreatedDTO dto)
        {
            if (!ModelState.IsValid) return View(dto);
            var student = _mapper.Map<Student>(dto);
            _db.Students.Add(student);
            _db.SaveChanges();
            return RedirectToAction("Index");
        }
    }
}
```

Step 2 — Create the two views

1. Views/StudentAdmin/Index.cshtml — @model List, table showing CourseCount per student
2. Views/StudentAdmin/Create.cshtml — form bound to StudentCreateDTO with validation spans, exactly like Phase 4
 1. Create student "Rafid Hasan" — Index shows CourseCount 0
 2. Enroll Rafid via Phase 8 cart? No — cart stores ids only; instead add an Enrollment row in SSMS and refresh: CourseCount becomes 1
 3. Confirm the Email column never appears in any form (by design)

Task 4 Verify Reverse Map and Update Path (20 min)

1. Add a temporary action that maps StudentCreateDTO to StudentUpdateDTO and returns its FullName as Content() — proves the ReverseMap line
2. Explain (in your notes) why Email is Ignore()d on the update map: the email is the stable identifier and must not drift

5. Common Errors and Solutions

Error	Likely Cause	Solution
AutoMapperConfigurationException at startup	Unmapped member	Add ForMember Ignore or a MapFrom for the offending member
Missing type mapping exception	Map not declared	Every used pair needs CreateMap in MappingProfile
IMapper not resolving	AddAutoMapper missing	Register with typeof(MappingProfile) before Build
CourseCount always 0	Enrollments not loaded	Include(s => s.Enrollments) before projecting
Over-posted Id after edit	Bound the entity instead of the DTO	Bind StudentUpdateDTO and map onto the tracked entity

6. Completion Checklist

#	Checklist Item	Done
1	Three DTOs created with per-operation fields	☐
2	MappingProfile declares Create, Update (with Ignore) and Read maps	☐
3	AddAutoMapper registered and app still boots	☐
4	StudentAdmin Index shows CourseCount computed by ForMember	☐
5	Create flow persists through Map(dto)	☐
6	ReverseMap verified with the temporary action	☐

7. Self-Assessment Questions

- Name two security reasons to bind a DTO instead of the entity.
- What does ForMember add that plain name-matching cannot?
- Why is the Read DTO free to contain computed fields?
- What does ReverseMap generate and when would you use it?
- Where does AutoMapper fail if a member is unmapped — build, startup, or request?

8. Further Reading

- AutoMapper documentation: <https://docs.automapper.org>
- DTO pattern: <https://learn.microsoft.com/aspnet/core/web-api/advanced/formatting>

End of Phase — next: Phase 10 locks the door — cookie authentication.

Phase 10 — Cookie Authentication and Custom Authorization

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 10 — Cookie Authentication and Custom Authorization
Syllabus Module	Module 10 — Session & Cookie-Based Authentication
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual — project-based practice
Project	MediCare — Hospital Management System
Prerequisite	Phase 9 completed — DTO refactor live with AutoMapper

1. Phase Objectives

By the end of this phase, you will be able to:

- Explain how cookie authentication proves identity on each request
- Configure the cookie scheme with LoginPath and expiry

- Sign users in with claims and a named ClaimsIdentity
- Write CustomAuthorizeAttribute implementing IAuthorizationFilter
- Protect pages by role and render a role-aware menu

2. Prerequisites

- Phase 9 completed — UniPortalMVC builds cleanly
- Theory class Module 10 (Authentication and Authorization) attended

3. Theory Recap

3.1 How the Cookie Flow Works

On successful login you build a ClaimsIdentity named after your scheme, attach claims (name, role), and call SignInAsync. ASP.NET Core serialises the claims into an encrypted cookie named UniPortalLogin. Every later request carries that cookie; the authentication middleware decrypts it and rebuilds HttpContext.User — no server-side session storage required for identity itself, though you will still mirror the role in session for the custom filter to read.

3.2 Scheme Configuration

Program.cs (authentication wiring)

```
builder.Services.AddAuthentication("CampusCookieAuth")
    .AddCookie(options =>
    {
        options.Cookie.Name = "UniPortalLogin";
        options.LoginPath = "/Account/Login";
        options.AccessDeniedPath = "/Account/AccessDenied";
        options.ExpireTimeSpan = TimeSpan.FromMinutes(30);
    });

var app = builder.Build();
app.UseRouting();
app.UseSession();
app.UseAuthentication(); // who are you?
app.UseAuthorization(); // are you allowed?
app.MapControllerRoute(...);
```

4. Phase Tasks

Warning: Important

Type all code manually. A one-line middleware disorder breaks login silently — test after every step.

Task 1 Configure the Cookie Scheme (15 min)

1. Add the AddAuthentication block before builder.Build()
2. Insert UseAuthentication and UseAuthorization in the order shown — authentication first, always

3. Keep UseSession before both, because the custom filter reads Session

Task 2 AccountController with Claims (35 min)

Step 1 — Create the controller

Controllers/AccountController.cs

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Mvc;

namespace UniPortalMVC.Controllers
{
    public class AccountController : Controller
    {
        private static readonly Dictionary<string, (string Pw, string Role)>
        _users = new()
        {
            ["admin@uniportal.edu"] = ("Admin@123", "Admin"),
            ["faculty@uniportal.edu"] = ("Faculty@123", "Faculty"),
            ["student@uniportal.edu"] = ("Student@123", "Student")
        };

        [HttpGet]
        public IActionResult Login(string returnUrl = "/")
        {
            ViewBag.ReturnUrl = returnUrl;
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Login(string email, string password,
            string returnUrl = "/")
        {
            if (!_users.TryGetValue(email, out var u) || u.Pw != password)
            {
                ModelState.AddModelError("", "Invalid email or password");
                return View();
            }

            var claims = new List<Claim>
            {
                new Claim(ClaimTypes.Name, email),
                new Claim(ClaimTypes.Role, u.Role)
            };
            var identity = new ClaimsIdentity(claims, "CampusCookieAuth");
            var principal = new ClaimsPrincipal(identity);

            HttpContext.SignInAsync("CampusCookieAuth", principal).Wait();
            HttpContext.Session.SetString("UserRole", u.Role);

            return LocalRedirect(returnUrl);
        }
    }
}
```

```

}

public IActionResult Logout()
{
    HttpContext.SignOutAsync("CampusCookieAuth").Wait();
    HttpContext.Session.Clear();
    return RedirectToAction("Index", "Home");
}

public IActionResult AccessDenied()
{
    return View();
}
}
}

```

Step 2 — Create the Login and AccessDenied views

1. Login.cshtml — email + password form with validation spans and a hidden returnUrl field
2. AccessDenied.cshtml — friendly message plus a link home

Task 3 CustomAuthorizeAttribute (30 min)

Step 1 — Build the filter

```

Security/CustomAuthorizeAttribute.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

namespace UniPortalMVC.Security
{
    public class CustomAuthorizeAttribute : Attribute, IAuthorizationFilter
    {
        private readonly string _role;

        public CustomAuthorizeAttribute(string role)
        {
            _role = role;
        }

        public void OnAuthorization(AuthorizationFilterContext context)
        {
            var user = context.HttpContext.User;

            if (!user.Identity.IsAuthenticated)
            {
                context.Result = new RedirectToActionResult(
                    "Login", "Account", null);
                return;
            }

            var sessionRole = context.HttpContext.Session
                .GetString("UserRole");

```

```
if (sessionRole != _role)
{
    context.Result = new RedirectToActionResult(
        "AccessDenied", "Account", null);
}
}
}
}
```

Note: Why read the role from session?

This mirrors your official manual: the cookie proves authentication, the session stores the role the filter compares. In production you would read `User.IsInRole(_role)` from the cookie claims — mention that in your viva.

Task 4 Protect Pages by Role (25 min)

Step 1 — Guard the admin area

Controllers/AdminController.cs

```
using Microsoft.AspNetCore.Mvc;
using UniPortalMVC.Security;

namespace UniPortalMVC.Controllers
{
    [CustomAuthorize("Admin")]
    public class AdminController : Controller
    {
        public IActionResult Dashboard()
        {
            ViewBag.Role = HttpContext.Session.GetString("UserRole");
            return View();
        }
    }
}
```

1. Create Views/Admin/Dashboard.cshtml showing ViewBag.Role
2. Create Views/Account/Login.cshtml and AccessDenied.cshtml
3. While logged out, open /Admin/Dashboard — redirected to /Account/Login
4. Log in as student@uniportal.edu — redirected to AccessDenied
5. Log in as admin@uniportal.edu — dashboard renders with Role = Admin

Task 5 Role-Aware Menu and Logout (20 min)

Step 1 — Make the navbar react to the login state

Views/Shared/_Layout.cshtml (navbar additions)

```
<ul class="navbar-nav flex-row gap-3">
    ...existing links...

    @if (User.Identity.IsAuthenticated)
```

```

{
<li class="nav-item">
<span class="nav-link text-warning">@User.Identity.Name</span>
</li>
<li class="nav-item">
<a class="nav-link" asp-controller="Account"
asp-action="Logout">Logout</a>
</li>
}
else
{
<li class="nav-item">
<a class="nav-link" asp-controller="Account"
asp-action="Login">Login</a>
</li>
}
</ul>

```

1. Login as faculty, confirm the navbar shows the email and Logout
2. Logout, confirm the Login link returns and /Admin/Dashboard redirects

5. Common Errors and Solutions

Error	Likely Cause	Solution
Redirect loop on login	Login action itself behind the filter	AccountController must carry no CustomAuthorize attribute
404 after sign-in	AccessDeniedPath or LoginPath typo	Paths must match controller/action casing exactly
Cookie never issued	UseAuthentication missing	Add it after UseSession and before UseAuthorization
Always AccessDenied	Session role lost after restart	Role lives in session; re-login after an app restart
SignInAsync hangs	Scheme name mismatch	The name passed to SignInAsync must equal "CampusCookieAuth"

6. Completion Checklist

#	Checklist Item	Done
1	Cookie scheme configured with UniPortalLogin and 30-minute expiry	☐
2	Middleware order: UseSession, UseAuthentication, UseAuthorization	☐
3	Login issues claims and mirrors the role in session	☐
4	CustomAuthorize redirects anonymous users to Login	☐
5	Wrong-role user lands on AccessDenied; correct role reaches Dashboard	☐

#	Checklist Item	Done
6	Navbar shows Login or email+Logout depending on state	☐

7. Self-Assessment Questions

- What exactly does SignInAsync write, and where does it live afterwards?
- Why must UseAuthentication run before UseAuthorization?
- What are the two checks inside OnAuthorization?
- Why does AccountController need no [CustomAuthorize] of its own?
- How would you switch the filter to read roles purely from claims?

8. Further Reading

- Cookie authentication: <https://learn.microsoft.com/aspnet/core/security/authentication/cookie>
- Authorization filters: <https://learn.microsoft.com/aspnet/core/mvc/controllers/filters>

End of Phase — next: Phase 11 opens the second front door — the Web API.

Phase 11 — Web API Basics: UniPortal.Api

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 11 — Web API Basics: UniPortal.Api
Syllabus Module	Module 11 — API Basics
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual — project-based practice
Project	MediCare — Hospital Management System
Prerequisite	Phase 10 completed — cookie auth and roles protecting the portal

1. Phase Objectives

By the end of this phase, you will be able to:

- Explain what makes an API controller different from an MVC controller
- Add a Web API project to the existing solution

- Scaffold UniversityDB into the API project
- Return proper status codes: 200, 201, 400, 404
- Test the endpoints with Swagger and Postman

2. Prerequisites

- Phase 6 completed — UniversityDB exists with the Department table
- Phase 10 completed — solution builds cleanly
- Postman installed (Phase 0)
- Theory class Module 11 (API Basics) attended

3. Theory Recap

3.1 MVC Controller vs API Controller

Aspect	MVC Controller	API Controller
Base class	Controller	ControllerBase
Attribute	none required	[ApiController] required
Returns	Views (ActionResult with HTML)	Data — JSON via Ok, CreatedAtAction, NotFound
Validation	ModelState checks in actions	Automatic 400 before your code runs
UniPortal use	Student and staff screens	Mobile app, integrations

3.2 Status Codes You Will Use Today

Code	Helper	Meaning in UniPortal.Api
200	Ok(...)	GET succeeded — departments returned
201	CreatedAtAction(...)	POST created — Location header points at the new row
400	BadRequest(...) / automatic	Invalid payload or business rule violated
404	NotFound()	Unknown department id
500	StatusCode(500, ...)	Unexpected exception caught in the action

4. Phase Tasks

Warning: Important

Type all code manually. The API project must compile after every task.

Task 1 Add UniPortal.Api to the Solution (15 min)

1. Right-click the UniPortal solution, Add, New Project
2. Choose **ASP.NET Core Web API**, Next
3. Name it UniPortal.Api, framework .NET 8
4. Authentication: None; Configure for HTTPS: ticked; **Use controllers: ticked**; OpenAPI: keep ticked; Create
5. Right-click the solution, Configure Startup Projects, set both projects to Start for later phases

Task 2 Scaffold UniversityDB into the API (25 min)

Step 1 — Packages and connection string

1. Set PMC Default project to UniPortal.Api and install the two EF packages exactly as in Phase 6
2. Copy the ConnectionStrings block into UniPortal.Api/appsettings.json — same name UniversityDb, same server

Step 2 — Scaffold with a distinct context name

Developer PowerShell (inside UniPortal.Api folder)

```
dotnet ef dbcontext scaffold "Name=UniversityDb" Microsoft.EntityFrameworkCore.SqlServer
--context UniversityApiDbContext --context-dir EF --output-dir EF/Tables --force
```

Note: Why --context UniversityApiDbContext?

The MVC project already owns UniversityDbContext. Renaming the scaffolded context keeps both projects independent and both compilable — you practise scaffolding the same database twice, once per consumer.

Task 3 Register Services and JSON Options (15 min)

UniPortal.Api/Program.cs

```
using Microsoft.EntityFrameworkCore;
using UniPortal.Api.EF;

builder.Services.AddDbContext<UniversityApiDbContext>(options =>
options.UseSqlServer(builder.Configuration
.GetConnectionString("UniversityDb")));

builder.Services.AddControllers()
.AddJsonOptions(o =>
o.JsonSerializerOptions.ReferenceHandler =
System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles);
```

1. Remove the OnConfiguring override inside UniversityApiDbContext (DI now supplies the options)
2. Confirm MapControllers() is already present before app.Run()

Task 4 DepartmentsController — Read + Create (35 min)

Step 1 — Type the controller

UniPortal.Api/Controllers/DepartmentsController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using UniPortal.Api.EF;
using UniPortal.Api.EF.Tables;

namespace UniPortal.Api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class DepartmentsController : ControllerBase
    {
        private readonly UniversityApiDbContext _db;

        public DepartmentsController(UniversityApiDbContext db)
        {
            _db = db;
        }

        // GET api/Departments
        [HttpGet]
        public IActionResult GetAll()
        {
            var list = _db.Departments
                .OrderBy(d => d.DepartmentName)
                .Select(d => new { d.DepartmentId, d.DepartmentName,
                d.OfficeRoom })
                .ToList();
            return Ok(list); // 200
        }

        // GET api/Departments/5
        [HttpGet("{id:int}")]
        public IActionResult GetById(int id)
        {
            var dept = _db.Departments.Find(id);
            if (dept == null)
            {
                return NotFound(new { message =
                $"Department {id} does not exist" }); // 404
            }
            return Ok(dept); // 200
        }

        // POST api/Departments
        [HttpPost]
        public IActionResult Create([FromBody] Department dto)
        {
            if (dto == null || string.IsNullOrEmpty(dto.DepartmentName))
            {
                return BadRequest(new { message =
                "departmentName is required" }); // 400 manual
            }
        }
    }
}
```

```

}

bool exists = _db.Departments.Any(d =>
d.DepartmentName == dto.DepartmentName);
if (exists)
{
return BadRequest(new { message =
$"Department '{dto.DepartmentName}' already exists" });
}

_db.Departments.Add(dto);
_db.SaveChanges();

return CreatedAtAction(nameof(GetById),
new { id = dto.DepartmentId }, dto); // 201
}
}
}

```

Tip: Automatic 400

Because the controller carries [ApiController], any [Required] failure on the bound model short-circuits to a 400 ProblemDetails before your action even runs. The manual BadRequest above catches the extra business rule the framework cannot know — both layers exist for a reason.

Task 5 Swagger and the Postman Collection (30 min)

1. Run UniPortal.Api (F5 on the API project) and open /swagger
2. Try GET api/Departments from Swagger — 200 with the three seeded rows
3. In Postman create a collection UniPortal API with these six requests

#	Request	Body	Expected
1	GET https://localhost:port/api/Departments	None	200, array of departments
2	GET https://localhost:port/api/Departments/1	None	200, single object
3	GET https://localhost:port/api/Departments/999	None	404 with message
4	POST https://localhost:port/api/Departments	{"departmentName": "Mathematics", "officeRoom": "B-501"}	201 with Location header
5	GET the Location header URL	None	200, Mathematics returned
6	POST again with empty departmentName	{"departmentName": ""}	400 with the model error

Warning: Postman body settings

Choose Body, raw, JSON — otherwise Postman sends text/plain and the framework answers 415 Unsupported Media Type.

5. Common Errors and Solutions

Error	Likely Cause	Solution
Swagger page blank or 404	UseSwagger calls missing	Keep AddSwaggerGen/UseSwagger/UseSwaggerUI in Development
Scaffold hits the wrong project	CLI run outside the API folder	cd into UniPortal.Api before scaffolding
JSON self-referencing loop	Navigation properties chase each other	Set ReferenceHandler.IgnoreCycles in AddJsonOptions
404 on api/Departments	MapControllers missing	Add app.MapControllers() before app.Run()
415 Unsupported Media Type	Postman body not set to JSON	Choose raw and JSON in the Postman Body tab

6. Completion Checklist

#	Checklist Item	Done
1	UniPortal.Api project added and starts standalone	☐
2	UniversityApiDbContext scaffolded; OnConfiguring removed	☐
3	Program.cs registers the context, IgnoreCycles, and Swagger	☐
4	GET all and GET by id return 200; unknown id returns 404	☐
5	POST returns 201 with a working Location header	☐
6	Invalid POST returns 400	☐
7	All six Postman requests recorded with their status codes	☐

7. Self-Assessment Questions

- What does [ApiController] give you that a plain MVC controller lacks?
- Why does CreatedAtAction exist instead of returning Ok after a POST?
- What produces the automatic 400 response, and when do you still write BadRequest manually?
- What is ReferenceHandler.IgnoreCycles protecting the serializer from?
- Why give the API its own scaffolded context name?

8. Further Reading

- Create web APIs with ASP.NET Core: <https://learn.microsoft.com/aspnet/core/web-api>
- Controller action return types: <https://learn.microsoft.com/aspnet/core/web-api/action-return-types>
- Get started with Swashbuckle and Swagger UI:
<https://learn.microsoft.com/aspnet/core/tutorials/getting-started-with-swashbuckle>

End of Phase — next: Phase 12 completes the API — full CRUD with DTOs and AutoMapper.

Phase 12 — Full CRUD API with DTOs and AutoMapper

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 12 — Full CRUD API with DTOs and AutoMapper
Syllabus Module	Module 12 — API Development CRUD
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual — project-based practice
Project	MediCare — Hospital Management System
Prerequisite	Phase 11 completed — UniPortal.Api serves departments over HTTP

1. Phase Objectives

By the end of this phase, you will be able to:

- Scaffold the Code-First UniPortalDb into the API project
- Design API DTOs with a computed member via AutoMapper
- Implement the complete CRUD set with correct status codes
- Guard business rules with manual 400s on top of automatic validation
- Complete a Postman matrix covering every verb and status

2. Prerequisites

- Phase 7 completed — UniPortalDb exists with Courses and Enrollments
- Phase 11 completed — API project runs with departments endpoints

3. Theory Recap

3.1 The CRUD-to-Status Contract

Operation	Route	Success	Failure
GET all	GET api/Courses	200 + array	N/A
GET one	GET api/Courses/2	200 + object	404
Create	POST api/Courses	201 + Location	400 (validation or duplicate code)
Update	PUT api/Courses/2	204 NoContent	404 / 400
Delete	DELETE api/Courses/2	204 NoContent	404

3.2 DTOs at the Boundary

The API never binds or returns entities. `CourseSaveDTO` shapes what a client may write; `CourseApiDTO` shapes what it may read, including a computed `EnrollmentCount` flattened by `AutoMapper` from the `Enrollments` navigation. This mirrors Phase 9 on the MVC side — same skill, new consumer.

4. Phase Tasks

Warning: Important

Type all code manually. One missing Include shows up as a zero in the response — verify every field in Postman.

Task 1 Scaffold UniPortalDb into the API (20 min)

Developer PowerShell (inside `UniPortal.Api` folder)

```
dotnet ef dbcontext scaffold "Name=UniPortalDb" Microsoft.EntityFrameworkCore.SqlServer
--context UniPortalApiDbContext --context-dir EF --output-dir EF/Tables --force
```

1. Register the context in `UniPortal.Api/Program.cs` with the `UniPortalDb` connection string, exactly like Phase 11 did for `UniversityApiDbContext`
2. Remove the `OnConfiguring` override the scaffold wrote

Task 2 API DTOs and Mapping Profile (25 min)

Step 1 — Create the DTOs

`UniPortal.Api/DTOs/CourseDTOs.cs`

```
using System.ComponentModel.DataAnnotations;

namespace UniPortal.Api.DTOs
{
    public class CourseApiDTO
    {

```

```

public int CourseId { get; set; }
public string CourseCode { get; set; }
public string Title { get; set; }
public int CreditHours { get; set; }
public int EnrollmentCount { get; set; }
}

public class CourseSaveDTO
{
    [Required, StringLength(10)]
    public string CourseCode { get; set; }

    [Required, StringLength(80)]
    public string Title { get; set; }

    [Range(1, 6)]
    public int CreditHours { get; set; }
}

```

Step 2 — Create the API mapping profile

UniPortal.Api/Profiles/MappingProfile.cs

```

using AutoMapper;
using UniPortal.Api.DTOS;
using UniPortal.Api.EF.Tables;

namespace UniPortal.Api.Profiles
{
    public class MappingProfile : Profile
    {
        public MappingProfile()
        {
            CreateMap<Course, CourseApiDTO>()
                .ForMember(d => d.EnrollmentCount,
                    o => o.MapFrom(s => s.Enrollments.Count));

            CreateMap<CourseSaveDTO, Course>();
            CreateMap<CourseSaveDTO, Course>()
                .ForMember(d => d.CourseId, o => o.Ignore());
        }
    }
}

```

UniPortal.Api/Program.cs

```

builder.Services.AddAutoMapper(typeof(MappingProfile));

```

Task 3 CoursesController — the Full CRUD Set (45 min)

UniPortal.Api/Controllers/CoursesController.cs

```

using AutoMapper;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

```

```
using UniPortal.Api.DTOS;
using UniPortal.Api.EF;
using UniPortal.Api.EF.Tables;
using UniPortal.Api.Profiles;

namespace UniPortal.Api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class CoursesController : ControllerBase
    {
        private readonly UniPortalApiDbContext _db;
        private readonly IMapper _mapper;

        public CoursesController(UniPortalApiDbContext db, IMapper mapper)
        {
            _db = db;
            _mapper = mapper;
        }

        // GET api/Courses
        [HttpGet]
        public IActionResult GetAll()
        {
            try
            {
                var courses = _db.Courses
                    .Include(c => c.Enrollments)
                    .OrderBy(c => c.CourseCode)
                    .ToList();
                var dto = _mapper.Map<List<CourseApiDTO>>(courses);
                return Ok(dto); // 200
            }
            catch (Exception ex)
            {
                return StatusCode(500, new { message =
                    "Unexpected error while loading courses",
                    detail = ex.Message }); // 500
            }
        }

        // GET api/Courses/2
        [HttpGet("{id:int}")]
        public IActionResult GetById(int id)
        {
            var course = _db.Courses
                .Include(c => c.Enrollments)
                .FirstOrDefault(c => c.CourseId == id);
            if (course == null)
            {
                return NotFound(new { message =
                    $"Course {id} does not exist" }); // 404
            }
            return Ok(_mapper.Map<CourseApiDTO>(course)); // 200
        }
    }
}
```

```
}

// POST api/Courses
[HttpPost]
public IActionResult Create([FromBody] CourseSaveDTO dto)
{
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState); // 400
    }

    bool duplicate = _db.Courses.Any(c =>
        c.CourseCode == dto.CourseCode);
    if (duplicate)
    {
        return BadRequest(new { message =
            $"Course code {dto.CourseCode} already exists" });
    }

    var course = _mapper.Map<Course>(dto);
    _db.Courses.Add(course);
    _db.SaveChanges();

    return CreatedAtAction(nameof(GetById),
        new { id = course.CourseId },
        _mapper.Map<CourseApiDTO>(course)); // 201
    }

// PUT api/Courses/2
[HttpPut("{id:int}")]
public IActionResult Update(int id, [FromBody] CourseSaveDTO dto)
{
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }

    var course = _db.Courses.Find(id);
    if (course == null)
    {
        return NotFound(new { message =
            $"Course {id} does not exist" }); // 404
    }

    _mapper.Map(dto, course); // map onto the tracked entity
    _db.SaveChanges();

    return NoContent(); // 204
    }

// DELETE api/Courses/2
[HttpDelete("{id:int}")]
public IActionResult Delete(int id)
{

```

```

var course = _db.Courses.Find(id);
if (course == null)
{
    return NotFound(new { message =
        $"Course {id} does not exist" }); // 404
}

_db.Courses.Remove(course);
_db.SaveChanges();

return NoContent(); // 204
}
}
}

```

Task 4 The Postman Matrix (30 min)

#	Request	Body	Expected
1	GET api/Courses	None	200; CSC4164 shows EnrollmentCount 1
2	GET api/Courses/1	None	200 with EnrollmentCount
3	GET api/Courses/999	None	404 with message
4	POST api/Courses	{"courseCode":"CSC1101","title":"Intro to Programming","creditHours":3}	201 + Location header
5	POST duplicate code CSC4164	{"courseCode":"CSC4164","title":"Dup", "creditHours":3}	400 duplicate message
6	POST bad payload	{"courseCode":"","title":"","creditHours":9}	400 automatic validation errors
7	PUT api/Courses/3	{"courseCode":"PHY1102","title":"Physics II (updated)","creditHours":4}	204; GET 3 shows the change
8	DELETE api/Courses/3	None	204; then GET 3 returns 404

Tip: Read the Location header

After request 4, copy the Location header URL into a new GET tab. It must return the created course with its new id — that is CreatedAtAction doing its job.

Task 5 Verify Auto 400 in Swagger (10 min)

1. Open /swagger, expand POST api/Courses, click Try it out
2. Submit an empty body — Swagger shows 400 with validation ProblemDetails before your code ran
3. Note the field errors array — that is [ApiController] automatic validation in the response shape

5. Common Errors and Solutions

Error	Likely Cause	Solution
AutoMapperMappingException on PUT	Mapping CourseId from dto	Ignore CourseId in the save map (already in the profile)
EnrollmentCount always 0	Missing Include	Include(c => c.Enrollments) before mapping
500 with DbContext disposed	Query executed after return	Materialise with ToList()/FirstOrDefault before mapping
405 Method Not Allowed	Wrong verb in Postman	Match PUT for update, DELETE for delete
Validation never triggers	[ApiController] attribute removed	Keep [ApiController] on the controller class

6. Completion Checklist

#	Checklist Item	Done
1	UniPortalApiDbContext scaffolded and registered	☐
2	API DTOs and MappingProfile with EnrollmentCount flattening	☐
3	GET all and GET one return 200 with computed counts	☐
4	POST returns 201 + Location; duplicate code and bad payload return 400	☐
5	PUT maps onto the tracked entity and returns 204	☐
6	DELETE returns 204 and the row disappears from SSMS	☐
7	All 8 Postman matrix requests recorded with status codes	☐

7. Self-Assessment Questions

- Why does Update return 204 instead of the modified object?
- What is the difference between the automatic 400 and your manual 400?
- Why must Update map onto the tracked entity instead of binding an entity?
- Where does EnrollmentCount come from when no column has that name?
- Which single line would you change to log and rethrow instead of returning 500?

8. Further Reading

- Tutorial: create a controller-based web API:
<https://learn.microsoft.com/aspnet/core/web-api/action-return-types>
- Handle errors in ASP.NET Core APIs: <https://learn.microsoft.com/aspnet/core/web-api/handle-errors>

End of Phase — next: Phase 13 — the capstone: everything, together.

Phase 13 — Capstone: Grand Integration and Demo

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Phase	Phase 13 — Capstone: Grand Integration and Demo
Syllabus Module	Capstone — integrates Modules 0 to 12
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual — project-based practice
Project	MediCare — Hospital Management System
Prerequisite	Phases 0-12 completed — the full UniPortal solution builds and runs

1. Phase Objectives

By the end of this phase, you will be able to:

- Run both projects together with one action
- Consume your own API from the MVC portal with `IHttpClientFactory`
- Execute the complete student registration journey end to end
- Pass a security review of the whole solution
- Demonstrate the system in five minutes without notes

2. Prerequisites

- All phase checklists ticked — Phases 0 to 12
- Both databases reachable in SSMS
- Postman collection from Phases 11 and 12 at hand

3. Integration Map

UniPortal now has two front doors over two databases. The portal is the human door: validated admission (Phases 4-5), department CRUD over the legacy UniversityDB (Phase 6), courses and enrollments over UniPortalDb (Phase 7), a session cart (Phase 8), DTO screens (Phase 9), login with roles (Phase 10). The API is the machine door: departments (Phase 11) and courses CRUD (Phase 12). The capstone wires the two doors together and proves the whole in one journey.

Layer	Pieces	Born In
UniPortalMVC	Controllers, Views, Validations, DTOs, Profiles	Phases 1-10
UniPortal.Api	Departments + Courses endpoints, DTOs, Swagger	Phases 11-12
UniversityDB	Department (legacy)	Phase 6
UniPortalDb	Course, Student, Enrollment + seed	Phase 7

4. Phase Tasks

Warning: Important

Type all code manually. Fix compile errors immediately — the demo depends on a green build.

Task 1 Run Both Projects with One F5 (15 min)

1. Right-click the solution, Properties, Multiple startup projects
2. Set UniPortalMVC and UniPortal.Api both to Start
3. F5 — both browser windows open; note each HTTPS port

Task 2 Consume the API from the Portal (35 min)

Step 1 — Register IHttpClientFactory

```
UniPortalMVC/Program.cs
builder.Services.AddHttpClient("api", client =>
{
    client.BaseAddress = new Uri("https://localhost:7nnn"); // your API port
});
```

Step 2 — Type the API-backed directory page

```
Controllers/CatalogController.cs
using System.Net.Http.Json;
using Microsoft.AspNetCore.Mvc;

namespace UniPortalMVC.Controllers
{
    public class CatalogController : Controller
    {
        private readonly IHttpClientFactory _http;

        public CatalogController(IHttpClientFactory http)
        {
            _http = http;
        }

        public async Task<IActionResult> Index()
```

```

{
    var client = _http.CreateClient("api");
    var courses = await client
        .GetFromJsonAsync<List<CourseRow>>("/api/Courses");
    return View(courses);
}

public class CourseRow
{
    public int CourseId { get; set; }
    public string CourseCode { get; set; }
    public string Title { get; set; }
    public int CreditHours { get; set; }
    public int EnrollmentCount { get; set; }
}

```

Step 3 — Create the view and the failure path

1. Views/Catalog/Index.cshtml — table of courses with EnrollmentCount
2. Wrap the GetFromJsonAsync call in try-catch: on failure show a warning banner "Course API unavailable" and render the local _db list instead
3. Stop the API project, refresh the page — banner appears, portal still works. Start the API again — live data returns

Tip: Why the failure path matters

A portal that dies when its API dies is not a system, it is a coin flip. Resilience with a graceful fallback is exactly what examiners probe.

Task 3 The Seven-Step User Journey (35 min)

Step	Action	Proves
1	Apply via /Admission/Apply with one deliberate invalid field	Phases 4-5 validation
2	Correct the field, submit, see the Success page	ModelState + PRG flow
3	Login as student@uniportal.edu	Phase 10 cookie auth
4	Browse /students and /course, add two courses to the cart	Phases 2-3 + 8
5	Confirm the cart total credits, remove one course	Session cart lifecycle
6	Open the API-backed /Catalog page	Phase 11-12 integration
7	Login as admin, open /Admin/Dashboard, verify SSMS rows	Phase 10 roles + Phase 6-7 data

1. Run the journey once completely, in order, without skipping

2. After step 7, check SSMS: the enrollment rows from the cart exist, the new department from any earlier test persists

Task 4 Security Pass (20 min)

Check	Expected
Anonymous /Admin/Dashboard	Redirects to /Account/Login
Student role on admin pages	Redirects to AccessDenied
Cookie flags	UniPortalLogin, CampusTheme HttpOnly; Secure in HTTPS
Server validation with JS off	Admission form still rejects bad data
API without claims	Courses endpoint open by design; note who should call it
SQL injection probe	Tag helpers and EF parameterise — nothing breaks

Task 5 Rehearse the Five-Minute Demo (15 min)

Minute	Show	Say
0:00-0:45	SSMS two databases	Database-First legacy and Code-First migrations side by side
0:45-1:30	Admission form with validation	Annotations plus custom rules from Phase 5
1:30-2:15	Login as student, browse routes	Attribute routing and the Razor layout from Phases 2-3
2:15-3:00	Cart add and remove	Session state from Phase 8
3:00-3:45	API catalog page, then Swagger	DTOs, AutoMapper, status codes end to end
3:45-4:30	Login as admin, dashboard, roles	Cookie auth and the custom authorization filter
4:30-5:00	Phase checklists recap	Fourteen green checklists = fourteen modules of proof

5. Common Integration Pitfalls

Symptom	Likely Cause	Solution
Catalog page shows warning banner	Wrong API base port	Copy the exact https port from the API browser window
SSL trust error MVC to API	Development certificate untrusted	Run dotnet dev-certs https --trust once
Enrollments not saved	Second context not registered in portal	UniPortalDbContext registration from Phase 7 must still exist

Symptom	Likely Cause	Solution
Demo runs but role links missing	Testing without logging in	The role-aware menu only renders for authenticated users
Two projects fight over one port	launchSettings collision	Each project keeps its own https port — leave defaults alone

6. Final Grand Checklist

#	Checklist Item	Done
1	Both projects run simultaneously from one F5	☐
2	Portal consumes the API through IHttpConnectionFactory with a failure path	☐
3	Cart converts into Enrollment rows visible in SSMS	☐
4	Seven-step user journey table executed once in full	☐
5	Security pass checklist completed	☐
6	Five-minute demo rehearsed end to end without notes	☐
7	All fourteen phase checklists from this guide are ticked	☐

7. Self-Assessment — the Whole Course

- Trace one course registration from form submit to SQL row, naming every layer it crosses.
- Why do MVC and API projects each own a scaffolded or code-first context?
- Where exactly do DTOs appear twice in the registration journey, and why?
- Which single mis-ordered middleware line would break both auth and session?
- What would break first if you deleted the MappingProfile — portal, API, or both?

8. Where to Go Next

Your syllabus continues into Modules 13 to 15 — three-tier architecture, the repository pattern with generic interfaces, and EF Core inside a three-tier solution. UniPortal is the perfect specimen to refactor: extract the controllers into service classes, wrap each DbSet in a generic repository, and split the solution into Presentation, Business, and Data projects. Everything you built in Phases 0 to 13 is exactly the code those modules will reorganise — keep it compiling and you are ready.

- ASP.NET Core best practices: <https://learn.microsoft.com/aspnet/core/fundamentals/best-practices>
- EF Core performance: <https://learn.microsoft.com/ef/core/performance>

End of Phase — next: You finished the guide — present UniPortal with confidence.